Daffodil International University
Department of Software Engineering

System Design Document

Unit 4: Delivery & Handover

TrustBazaar: A Verified Peer-to-Peer Auction and Marketplace Platform for Used Goods

S. M. Nihal Ahmed (232-35-002)
Afrim Hossen Khan (232-35-076)
Sabikun Nahar Sinthia (232-35-475)

Status	Draft for team review
Version	0.1, 18 October 2026
Implements	SRS v2.0 (`docs/srs.tex`)
Depends on	Unit 1 (Identity & Access), Unit 2 (Order, Payment & Escrow), Unit 3 (Auctions, Listings & Search)
Database schema	`docs/design/sql/04_delivery.sql`, tested on PostgreSQL (90 checks and concurrency tests, Appendix B)

Contents

1 Introduction **3**
1.1 Purpose . . . 3
1.2 Scope . . . 3
1.3 Requirements covered . . . 3

2 Unit Overview **4**
2.1 Responsibilities . . . 4
2.2 Components . . . 4
2.3 Convention DC-14 . . . 5

3 Interfaces **5**
3.1 HTTP API . . . 5
3.2 Ports provided . . . 6
3.3 Courier provider port . . . 6
3.4 Events . . . 7
3.5 Ports required . . . 7

4 Data Design **8**
4.1 Tables and rules the database enforces . . . 8

5 Choosing a Method and Pricing the Courier **9**
5.1 Distance and the recommended method . . . 9
5.2 Zones . . . 9
5.3 Billed weight . . . 9
5.4 Fare table . . . 10
5.5 Quotes . . . 10

6 Meetup Handover **10**
6.1 Overview . . . 10
6.2 Suggesting a place . . . 12
6.3 Agreeing a time . . . 12
6.4 The handover code . . . 12
6.5 Deadline . . . 13

7 Courier Delivery **14**
7.1 Addresses . . . 14
7.2 Booking . . . 14
7.3 Shipment status . . . 14
7.4 Applying courier status reports . . . 14

7.5 Pickup deadline . . . 15
7.6 Cancellation . . . 15
7.7 Failed delivery, return and non-delivery . . . 16
7.8 Return shipments . . . 16
7.9 The mock courier . . . 16
7.10 Operator issues . . . 16

8 Scheduled Jobs **17**

9 Security and Privacy **17**
9.1 Threats considered . . . 17

10 Parameters **18**

11 Testing **19**

12 Notes for Integration **19**

A Traceability **20**

B Database Schema and Tests **21**
B.1 Test results . . . 21
B.2 Schema . . . 22

1 Introduction

1.1 Purpose

This document is the fourth unit of TrustBazaar's System Design Document. It covers how a paid physical item reaches the buyer. There are two ways:

- **a meetup handover** at a safe public place, proved by a one-time code that the buyer gives the seller after inspecting the item;
- **courier delivery** at a fare fixed at checkout, tracked through the courier's status updates.

The unit records the two facts on which Unit 2's escrow depends: the *fulfilment step* (handover, or courier pickup), whose deadline decides whether the buyer is refunded, and the *delivery event*, from which the dispute window and the auto-release timeout are counted (SRS FR-PAY-07, FR-PAY-08). Acceptance criterion 3 (money is never released before the item is delivered) therefore depends on this unit recording these facts truthfully.

The unit follows conventions DC-1 to DC-13 (Unit 1 Section 2; Unit 2 Section 2.3) and adds DC-14 (Section 2.3). Game-account handover is not part of this unit; it is Unit 5.

1.2 Scope

In scope:

- the distance between buyer and seller, the recommended method, and the delivery zones;
- courier fares, the versioned fare table, and quotes fixed at checkout;
- curated safe meetup points and the suggestion of points near the midpoint;
- meetup proposals and agreement, the handover code, the wrong-entry lock and the operator unlock;
- courier booking, exact addresses, provider webhooks and status queries, the shipment status machine, failed delivery and return to the seller;
- return shipments ordered by a dispute decision (FR-DSP-06(d));
- the mock courier and its labelling as simulated;
- the fulfilment records used in a dispute case file (FR-DSP-05).

Out of scope, owned by other units:

- escrow state, deadlines, refunds and the split of the delivery fee (Unit 2; this unit reports steps and events through `EscrowCommands`);
- the listing's parcel classes, fulfilment options and location (Unit 3, read through its port);
- credential handover and takeover for game accounts (Unit 5);
- delivering notifications and deadline reminders (Unit 6; this unit emits events);
- dispute decisions and the operator queues (Unit 7; this unit supplies the records, the issues and the return booking).

1.3 Requirements covered

- **Functional:** FR-DLV-01 to FR-DLV-14; FR-PAY-08 and FR-PAY-09 (fulfilment side); FR-DSP-01 (when a `not_received` dispute may open), FR-DSP-05 (delivery and handover records) and FR-DSP-06(d) (return by courier); FR-NTF-07 (the events behind meetup and pickup reminders); FR-PRV-01 (the courier as a recipient of personal data).
- **Security:** SEC-PAY-03, SEC-PAY-04, SEC-PAY-05 and SEC-PAY-07, applied to the courier; SEC-DAT-07 (the code and addresses never logged); SEC-ACC-05 (masked addresses for staff); SEC-INP-05; SEC-ABU-01.

- **Other:** IF-SW-04, IF-COM-03; NFR-REL-04 (courier outage); NFR-CMP-05 (simulated bookings); DR-06.

2 Unit Overview

2.1 Responsibilities

1. Offer the buyer the methods the seller allows, recommend one by distance, and fix a courier fare at checkout from a published fare table.
2. Let buyer and seller agree a time at a safe public place without revealing where either of them lives.
3. Record a meetup handover only when the buyer has released the code, and stop guessing of the code.
4. Book the courier, keep exact addresses confidential, and turn the courier's status reports into exactly one fulfilment step and one delivery event, however often and in whatever order the reports arrive.
5. Hand every case that a rule cannot decide (a locked code, a pickup after cancellation, a report that makes no sense) to an operator rather than guessing.

2.2 Components

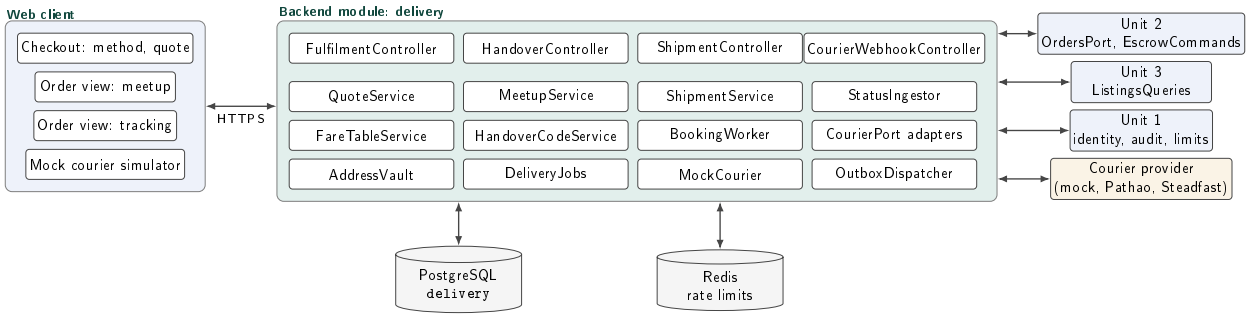


Figure 1: Components of the Delivery & Handover unit. The mock courier runs inside the backend, behind the same `CourierPort` as a real provider, so no extra container is needed (NFR-MNT-01).

Component	Responsibility
QuoteService	Distance, recommended method, zones, billed weight and fare; issues and claims quotes (Section 5).
FareTableService	Publishes new fare-table versions (administrator, audited).
MeetupService	Suggests meetup points; proposals, counter-proposals and agreement (Section 6).
HandoverCodeService	Derives the buyer's code; compares the seller's entry in constant time; records it (Section 6.4).
ShipmentService	Creates shipments from paid courier orders and return decisions; cancellation; addresses.
BookingWorker	Books with the provider, with an idempotency key and retries (Section 7.2).
StatusIngestor	Verifies webhooks, confirms money-moving statuses by query, applies statuses once (Section 7.4).
CourierPort adapters	One adapter per provider: `MockCourier` (default, C-3), Pathao and Steadfast sandboxes when access is granted.
AddressVault	Encrypts and decrypts exact addresses; erases them on schedule (Section 7.1).
DeliveryJobs	The scheduled jobs of Section 8.

Component	Responsibility
MockCourier	Simulated provider: booking API, status API, signed webhooks, and a simulator page for demonstrations (Section 7.9).

2.3 Convention DC-14

DC-14. When a delivery action must change escrow, the service first locks the order through `OrdersPort.lockForFulfilment(orderId, tx)`, then the delivery row, and then calls `EscrowCommands` in the same transaction (DC-13). This extends DC-13's lock order to: listing, order, *delivery row*, escrow, attempt. The database functions of this unit lock only delivery rows, and receive every order or listing fact they need as an argument. Under DC-1 they never read another unit's tables.

The rule matters for races between a person and a scheduled job. Recording a handover and Unit 2's fulfilment-deadline job both lock the order first, so one of them waits for the other and then sees its result. A handover entered at the deadline is therefore either recorded before the refund, or refused after it, and never both.

3 Interfaces

3.1 HTTP API

All paths are under `/api/v1`; policies and rate limits are those of Unit 1. A party sees the other party's district only, never their coordinates or address (FR-DLV-07, FR-ACC-10). The handover code appears in exactly one response: the buyer's `GET /orders/{id}/handover`.

Method	Path	Policy	Purpose (requirement)
Choosing a method and pricing			
`GET`	`/delivery/areas`	Public	Drop-off areas by district, in Bangla and English
`GET`	`/listings/{id}/fulfilment-options?areaId=`	Verified(buy)	Allowed methods, distance in whole km, recommended method, indicative courier fee (FR-DLV-01, 02, 09)
`POST`	`/delivery/quotes`	Verified(buy)	Binding quote for a listing, quantity and saved drop-off address; valid DV-23 (FR-DLV-08)
`GET, POST, DELETE`	`/me/addresses`	SignedIn	The user's own saved pickup and drop-off addresses
Meetup handover			
`GET`	`/orders/{id}/handover`	Buyer or seller	Status, deadline, suggested points, proposals, the agreed point and time. Buyer only: the code and the instruction. Seller only: attempts left
`POST`	`/orders/{id}/meetup-proposals`	Buyer or seller	Propose a point and time; replaces an open proposal (FR-DLV-04)
`POST`	`/orders/{id}/meetup-proposals/{pid}/accept`	The other party	Agree (FR-DLV-04)
`POST`	`/orders/{id}/meetup-proposals/{pid}/decline`	The other party	Decline
`POST`	`/orders/{id}/meetup-proposals/{pid}/withdraw`	Proposer	Withdraw an open proposal
`POST`	`/orders/{id}/handover-code`	Seller	Enter the buyer's code; a correct code is the delivery event (FR-DLV-06)

Method	Path	Policy	Purpose (requirement)
Courier			
GET	/orders/{id}/shipments	Buyer or seller	Status and history, the "simulated" label, and the caller's own address only (FR-DLV-10, 14)
PUT	/orders/{id}/shipment/pickup-address	Seller	Give the pickup address while the shipment is preparing
POST	/orders/{id}/return/received	Seller	Confirm that a returned item has arrived (FR-DSP-06(d))
POST	/webhooks/courier/{provider}	Provider signature	Status callbacks (IF-COM-03, Section 7.4)
Staff			
GET	/staff/delivery/issues	Staff(operator)	Open delivery issues (Section 7.10)
POST	/staff/delivery/issues/{id}/resolve	Staff(operator)	Resolve with a reason
POST	/staff/orders/{id}/handover/unlock	Staff(operator)	Issue a new code after a lock, with a reason
GET	/staff/orders/{id}/fulfilment-record	Staff(operator), case	Fulfilment record for a case; addresses masked unless a reason is given, and each display is logged (SEC-ACC-05)
POST, PATCH	/admin/meetup-points, /admin/areas	Staff(admin)	Maintain the curated points and areas
POST	/admin/fare-tables	Staff(admin)	Publish a new fare-table version (CP-43)
Mock courier (sandbox builds only; Section 7.9)			
POST	/mock-courier/consignments/{c}/advance	Staff(operator)	Move a simulated parcel to its next status, for demonstrations

Refusals return 422 with a stable code: courier_not_offered, parcel_too_large, quote_expired, quote_mismatch, meetup_time_out_of_range, proposal_closed, code_locked, handover_closed or deadline_passed. A wrong code returns 422 wrong_code with the number of attempts left. The response is the same whichever digits were wrong, so it gives no hint about the right code.

3.2 Ports provided

```
interface DeliveryPort {                          // used by Unit 2
  claimQuote(q: { quoteId: string; orderId: string; buyerId: string; listingId: string;
                  quantity: number }, tx: EntityManager): Promise<bigint>;   // the fee; one use only
  syncPickup(orderId: string): Promise<void>;     // query the courier before a missed-pickup refund
}
interface DeliveryCommands {                      // used by Unit 7
  quoteReturn(orderId: string, fromAreaId: number, toAreaId: number): Promise<Quote>;
  bookReturn(orderId: string, quoteId: string, payer: 'buyer' | 'seller',
             tx: EntityManager): Promise<string>;           // FR-DSP-06(d); returns the shipment id
  unlockHandoverCode(orderId: string, staffId: string, reason: string): Promise<void>;
  resolveIssue(issueId: string, staffId: string, resolution: string): Promise<void>;
}
interface DeliveryQueries {                       // used by Units 6 and 7
  fulfilmentRecord(orderId: string): Promise<FulfilmentRecord>;  // FR-DSP-05
  notReceivedAllowed(orderId: string): Promise<boolean>;         // FR-DLV-11, FR-DSP-01
  openIssues(cursor?: string): Promise<Page<DeliveryIssue>>;
}
```

3.3 Courier provider port

Each provider is an adapter behind one interface, chosen by configuration. The mock is the default (C-3, IF-SW-04).

```ts
interface CourierPort {
  book(p: { shipmentId: string; pickup: Address; dropoff: Address;
            parcel: { billedKg: number; sizeClass: SizeClass }; cashToCollect: 0 }): Promise<{ consignmentId:
      string }>;
  cancel(consignmentId: string): Promise<'cancelled' | 'already_picked_up'>;
  requestReturn(consignmentId: string): Promise<void>;
  status(consignmentId: string): Promise<{ status: ShipmentStatus; occurredAt: Date; eventId: string;
                                           returnCharge?: bigint }>;
  verifyWebhook(headers: Headers, rawBody: Buffer): WebhookEvent;     // throws if not authentic
}
```

- **Idempotent booking.** The shipment id is sent as the merchant order reference. A retried booking therefore finds the existing consignment instead of creating a second one (SEC-PAY-07).
- **No cash on delivery.** `cashToCollect` is always zero, because the item is already paid for (FR-DLV-05). Bangladeshi courier bookings are usually cash-on-delivery, so the adapter sets the amount to zero explicitly rather than relying on the provider's default.
- **Fixed endpoints.** Each adapter calls only its provider's configured base URL. No address supplied by a user or a webhook is ever fetched (SEC-INP-05).
- **Separate credentials** for sandbox and production. A sandbox build refuses production keys, and the reverse (SEC-PAY-05).

3.4 Events

Event	Use
Published (outbox, DC-6)	
`delivery.meetup.proposed`, `.agreed`	Notifications to the other party, with the point and time.
`delivery.handover.completed`	Notifications. The escrow change is already made in the same transaction (DC-14).
`delivery.handover.locked`	Notifications to both parties (FR-DLV-06); the operator queue.
`delivery.shipment.status_changed`	Notifications to both parties (FR-DLV-10); labelled as simulated where it is.
`delivery.return.delivered`	Disputes: the seller has the item back, so the refund under FR-DSP-06(d) can be made.
Consumed (idempotent)	
`orders.order.paid`	Self-pickup: create the handover with the order's deadline. Courier: create the shipment and queue the booking. The payload must carry the fulfilment method, the buyer, the seller, `fulfilment_due_at` and the quote id (I-19).
`orders.escrow.refunded` (before the delivery event)	Cancel the handover. Cancel the booking, or, if the courier already has the parcel, request a return (Section 7.6).
`identity.account.closed`	Delete the user's saved addresses (they are needed only for future orders). Shipment copies follow DV-30.

3.5 Ports required

Port	Provider and use
`IdentityQuery` (Unit 1)	`assertCanTransact(buy)`; `getVerifiedPhone`, passed to the courier as the contact number; new `getApproxArea`, the user's saved district and rounded coordinates (I-24).
`AuditService`, `RateLimiter`, `IdentityCrypto` (Unit 1)	Audit of staff actions and fare changes; limits on code entry and quotes; AES-GCM field encryption for addresses.

Port	Provider and use
`OrdersPort` (Unit 2)	New `lockForFulfilment`: locks the order row and returns its status, fulfilment, parties and deadline (DC-14, I-20).
`EscrowCommands` (Unit 2)	`recordFulfilmentStep`, `recordDeliveryEvent`, `refundCourierReturned`; new `recordCourierCost` (I-21).
`ListingsQueries` (Unit 3)	New `fulfilmentFacts(listingId)`: fulfilment options, weight and size class, district and rounded coordinates (I-23).
`ConfigPort`, `Clock`	CP and DV values; the server clock (DC-9, DC-10).

4 Data Design

The schema is in `docs/design/sql/04_delivery.sql` (Appendix B), loaded after Units 1 to 3. The unit owns one schema, `delivery`. Its foreign keys point only to earlier units: users and staff (Unit 1), orders (Unit 2) and listings (Unit 3), as DC-5 allows. It also calls Unit 3's pure function `listings.distance_km`, which reads no table.

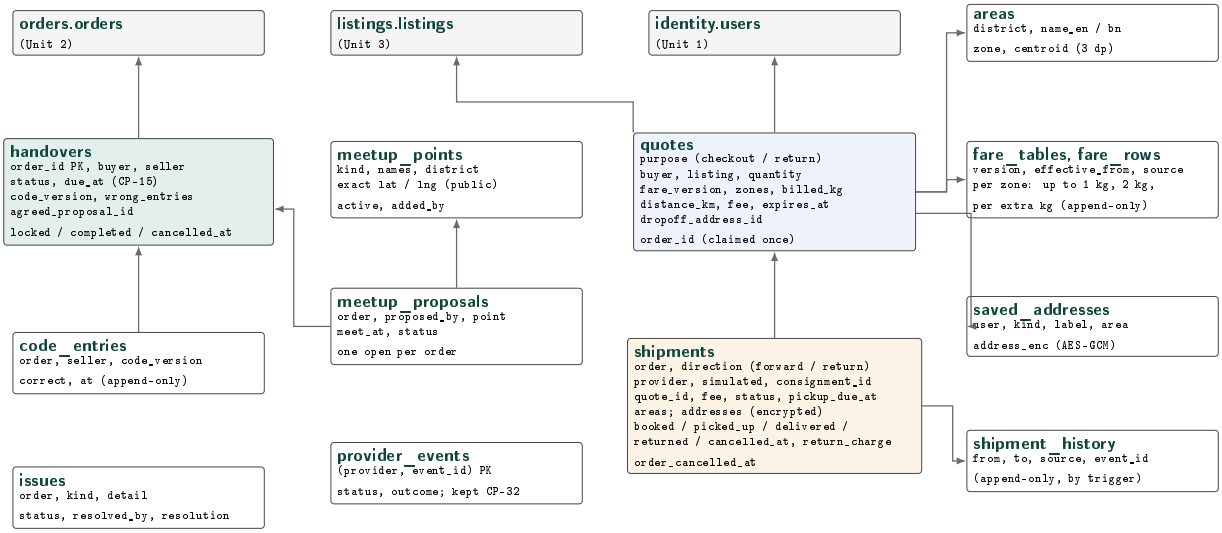


Figure 2: The `delivery` schema. Shipments, quotes and issues also reference `orders.orders`; those arrows are omitted for legibility. A shipment's quote must be the one claimed for the same order, with the same fee (trigger).

4.1 Tables and rules the database enforces

Table	Rules enforced by the database
`areas`	Unique per district; centroid inside Bangladesh, rounded to 3 decimal places.
`fare_tables`, `fare_rows`	Append-only. The fare never falls with weight. A version is used only once it has all three zones. Every version after the seed names the administrator who published it.
`quotes`	Fixed once issued. Claimed by one order only, and only by the buyer, listing and quantity it was issued for, before it expires. One checkout quote and one return quote per order. A checkout quote has the buyer's own drop-off address, in the quoted area.
`meetup_points`	Coordinates inside Bangladesh. Only active points can be proposed.

Table	Rules enforced by the database
`handovers`	Starts as `arranging`. Parties and deadline fixed. Only the permitted status changes. A completed or cancelled handover is closed. The wrong-entry count never falls, and the code version never changes, except when a locked code is unlocked, which issues the next version and resets the count.
`meetup_proposals`	Cannot be edited. One open proposal per order. Only the permitted status changes.
`code_entries`	Append-only.
`saved_addresses`	The exact address is stored only encrypted; deleting erases it.
`shipments`	Starts as `preparing`, with its own claimed quote and fee and the quoted drop-off area. One forward and at most one return shipment per order. Only the permitted status changes. Delivered and returned only when confirmed by a status query. Mock bookings are always simulated. Booking needs a consignment id and both addresses. Consignment id, addresses, quote and fee are fixed once booked. Time fields match the status. A return charge only on a return. Addresses are erased only from a final shipment, and nothing else in a final shipment can change.
`shipment_history`	Written by trigger on every status change; append-only.
`provider_events`	One row per provider event id, so a repeated event has no effect.
`issues`	One open issue of a kind per order. Resolution needs a staff member and a reason.

5 Choosing a Method and Pricing the Courier

5.1 Distance and the recommended method

At checkout the buyer chooses a drop-off area (or a saved address, which has one). The distance is computed between the listing's rounded coordinates and the area's centroid, with the haversine function of Unit 3, and shown in whole kilometres (FR-DLV-01). The recommended method is self-pickup when the distance is at or below CP-14 (2 km) and the seller allows it, and courier otherwise. The buyer may choose any method the seller allows. Self-pickup adds no fee (FR-DLV-02), which Unit 2's `fee_only_for_courier` constraint already enforces. `delivery.recommended_method` is the reference implementation of this rule, and its boundary at exactly 2 km is tested.

5.2 Zones

CP-43 prices by three zones: inside Dhaka, Dhaka suburbs, and outside Dhaka. Each area in the curated `areas` table has a zone. The seller's side has only a district and rounded coordinates, so its zone is that of the nearest listed area in the same district. A district with no listed areas is outside Dhaka. This handles the Dhaka district correctly, because it contains both city areas (Dhanmondi) and suburbs (Savar).

The zone of a route combines both ends. Both inside Dhaka gives inside Dhaka. Both within Dhaka or its suburbs gives suburbs. Anything else is outside Dhaka.

5.3 Billed weight

A listing has a weight class and a size class (FR-LST-03). Couriers charge bulky parcels by volume as well as weight, so each size class has a minimum billed weight (DV-25): small 0 kg, medium 2 kg, large 5 kg. Each unit is billed at the larger of the upper bound of its weight class and that minimum, and the order is billed at that times the quantity. Above DV-24 (10 kg), and for the `over_10kg` and `oversize` classes, courier is not offered: the item is self-pickup only, or the buyer orders fewer units at a time.

5.4 Fare table

Zone	Up to 1 kg	Up to 2 kg	Each further kg
Inside Dhaka	BDT 50	BDT 65	BDT 15
Dhaka suburbs	BDT 80	BDT 95	BDT 15
Outside Dhaka	BDT 100	BDT 120	BDT 20

The seed values are RedX's published merchant rates, one of the rate cards named in SRS reference R14. They were taken from a merchant's published shipping page that lists them (last updated May 2026), and should be confirmed against RedX's current merchant terms. As a cross-check, Paperfly publishes 70, 110 and 130 for the same zones, plus BDT 20 per further kilogram. The seed is therefore within the market range, which is what the SRS assumes ("reasonable approximations of published rate cards"). The administrator reviews it before the demonstration.

The table is versioned and append-only. A change is a new version with an effective time, a source and the administrator's name, recorded in the audit log. Every quote records the version it used (FR-OPS-08), so an old order can always be explained.

5.5 Quotes

1. The order page shows an *indicative* fee for the chosen area. It is computed the same way, but not stored.
2. When the buyer picks courier, `POST /delivery/quotes` issues a binding quote. It is tied to the buyer, the listing, the quantity and the buyer's saved drop-off address, and is valid for DV-23 (30 minutes, longer than the CP-13 checkout hold).
3. Unit 2 claims the quote while creating the order, in the same transaction (DC-13). The claim checks buyer, listing, quantity and expiry, and that the quote is unused, then fixes the fee on the order. One quote can serve only one order; the concurrency test raced ten orders for one quote and exactly one succeeded.
4. The buyer sees one total, item plus delivery, and never chooses a courier company (FR-DLV-09).

Worked example. A phone (up to 1 kg, small) listed in Dhanmondi, to be delivered to Savar, is billed at 1 kg. Dhanmondi is inside Dhaka and Savar is a suburb, so the route is in the suburbs zone and the fee is BDT 80. With the 10% delivery margin (CP-26), Unit 2's split later gives the platform BDT 8 and the courier BDT 72 (FR-DLV-13). A large item up to 2 kg sent outside Dhaka is billed at 5 kg (the size minimum), so its fee is $120 + 3 \times 20 =$ BDT 180.

Distance in the fare. FR-DLV-08 lists distance among the fare's inputs. The cited rate cards price by zone and weight only, so distance enters the fare through the zone. The quote still records the distance, which is shown to the buyer and used for the recommendation. A clarification of FR-DLV-08 is proposed in I-28.

6 Meetup Handover

6.1 Overview

When Unit 2 reports a self-pickup order paid, this unit creates the handover with the order's deadline, CP-15 (7 days after payment). From then on the order view shows both parties that the item is already paid for and that no cash is to change hands (FR-DLV-05). No cash option exists anywhere in the flow.

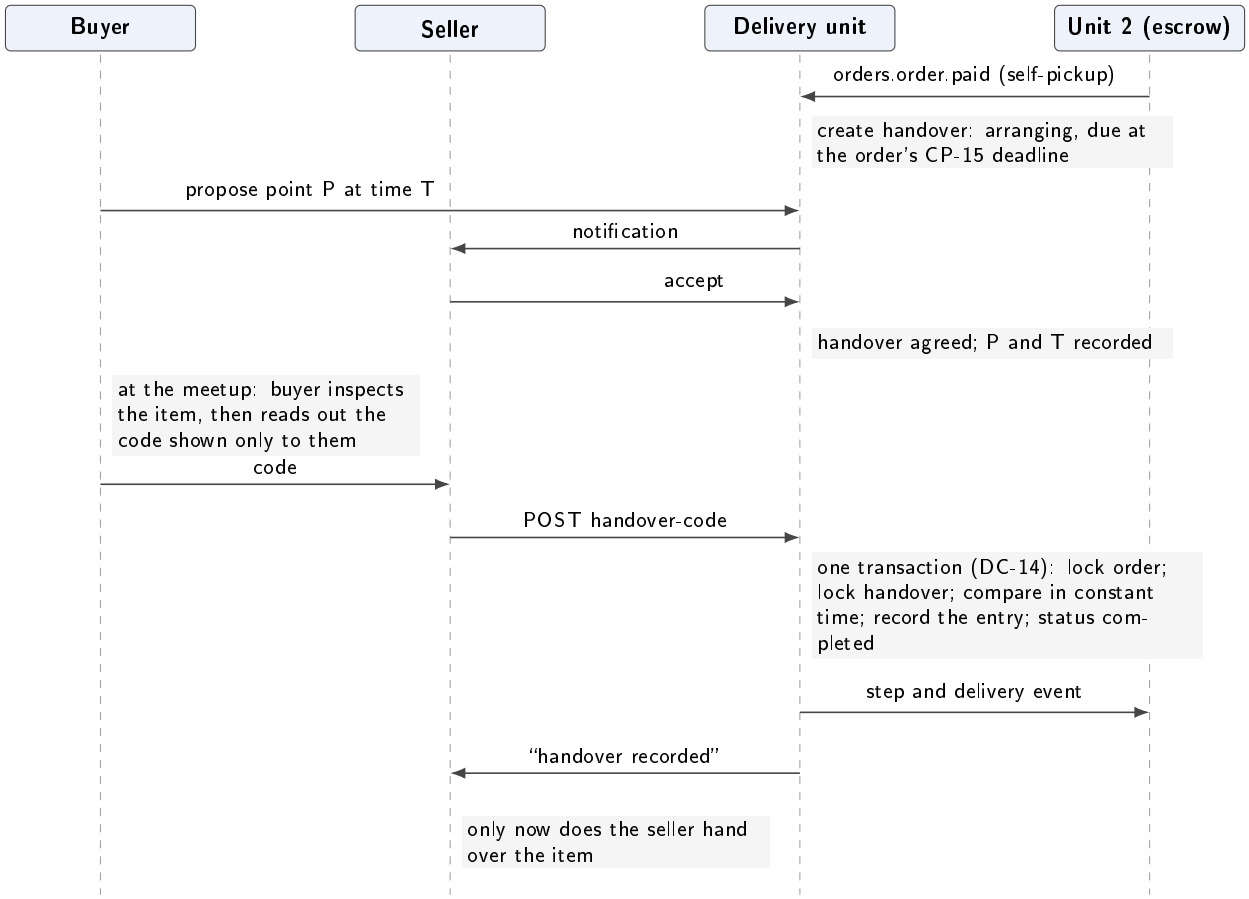


Figure 3: Meetup handover (FR-DLV-04 to FR-DLV-06). The code entry and the escrow's delivery event commit together, or not at all.

6.2 Suggesting a place

The order view suggests the DV-27 (three) active curated points nearest the midpoint of the two parties' locations (FR-DLV-03). The points are police stations, shopping-mall entrances, bank branches and similar public places, maintained by an administrator. They are public, so their exact location is shown, with an "open in maps" link built from their coordinates.

The parties' own locations stay approximate (FR-DLV-07), and the midpoint needs care. A party who knew their own location and the exact midpoint could compute the other's location as twice the midpoint minus their own. Each location is therefore snapped to a coarse grid of DV-27 (0.02 degrees, about 2 km) before the midpoint is taken. The API returns only the points and their distance from the caller, never the midpoint or distances from it. The test moves one party by about 300 m and confirms that the suggestions and their order do not change.

6.3 Agreeing a time

Either party proposes a point and a time; the other accepts or declines (FR-DLV-04). The rules, enforced in `delivery.propose_meetup` and `respond_meetup`, are:

- only the buyer and the seller can propose, and only the other party can answer;
- the time must be at least DV-26 (1 hour) ahead, and at least DV-26 (2 hours) before the handover deadline, so that a meeting cannot be planned that the deadline would cut short;
- a new proposal from either side replaces the open one, so there is never more than one open proposal (tested with 20 simultaneous proposals: one open, 19 superseded);
- a proposal cannot be edited, only replaced;
- a proposal whose time has passed lapses instead of being accepted;
- rescheduling keeps the existing agreement in force until the new time is accepted.

Proposals outside daylight hours are allowed, but the order view shows a safety notice. Agreement is not required for the handover itself: what proves the handover is the code, not the plan.

6.4 The handover code

What the parties see. The buyer's order view shows a 6-digit code with the instruction: *"Give this code to the seller only after you have inspected the item. Giving the code confirms that you have received it."* The seller's view shows an entry box and the instruction: *"Hand over the item only after this page shows Handover recorded."* The second instruction protects the seller from a buyer who takes the item and walks away during a slow network request.

Derivation, not storage. The code is never stored. It is computed on each request as

$$code = \text{HMAC-SHA-256}(HANDOVER_KEY,\ orderId \,\|\, : \,\|\, codeVersion) \bmod 10^6,$$

taking the first 8 bytes of the MAC as an unsigned integer, padded to 6 digits (DV-28). The reduction modulo 10^6 makes some codes more likely than others by a relative amount below 10^{-13}, which is negligible. Deriving the code has three benefits:

- a copy of the database reveals no codes;
- the buyer can reload the page and always see the same code;
- issuing a new code is just incrementing `code_version`.

`HANDOVER_KEY` is a new secret in Unit 1's key table (I-24). It is rotated only on compromise; a rotation changes every outstanding code, and both parties of each open handover are told to reload.

Checking an entry. The service derives the expected code and compares it with the entry in constant time, then calls `delivery.record_code_entry` with the result, inside the DC-14 transaction. The function:

- locks the handover;
- refuses anyone but the seller, a locked or closed handover, and any entry after the deadline;
- logs the entry with its code version;
- on a correct entry, completes the handover;
- on a wrong entry, increments the count, and at CP-45 (5) locks code entry, opens a `code_locked` issue and notifies both parties.

On a correct entry the service records the fulfilment step (`meetup_handover`) and the delivery event (source `handover_code`) through `EscrowCommands` in the same transaction, so the escrow's dispute window and auto-release time start at that moment (FR-PAY-07, FR-PAY-08).

Guessing. With 5 attempts at a 6-digit code, the chance of guessing it is $5/10^6$. The limit holds under concurrency: 20 simultaneous wrong entries produced exactly 4 "wrong", 1 "locked" and 15 refusals, with 5 entries logged. Ten simultaneous correct entries produced one handover and nine refusals. Code entry is also rate-limited per seller (DV-28: 10 per minute), and a wrong code's response is the same whatever was entered.

Unlock. A locked code stays locked until an operator, having contacted both parties, unlocks it with a reason (audited). Unlocking issues the next code version and resets the count; the old code no longer works. If nobody resolves the lock, the CP-15 deadline refunds the buyer as usual.

Secrecy. The code is never written to logs, analytics, URLs or notifications (SEC-DAT-07), and in particular not to a push notification that could appear on a locked screen. It is returned only to the buyer, and only over the authenticated API.

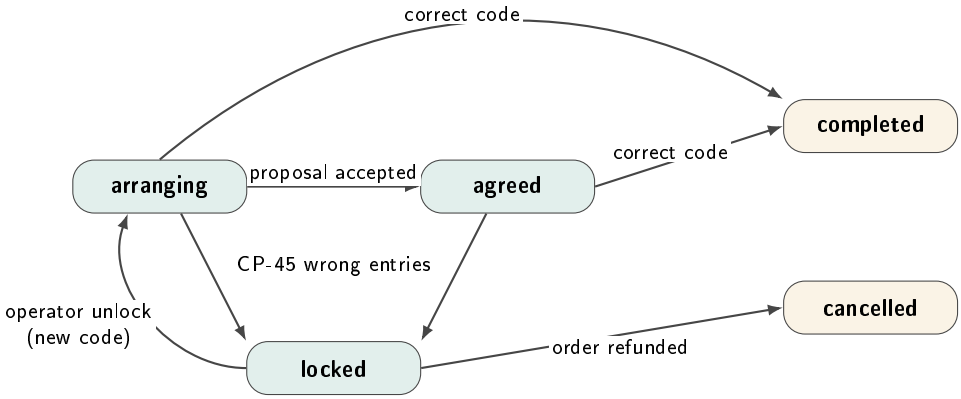


Figure 4: Handover status. Terminal states are shaded. An unlock returns to `agreed` if an agreement exists, otherwise to `arranging`. `arranging` and `agreed` also go to `cancelled` when the order is refunded (not drawn). The trigger `handover_guard` refuses every other change.

6.5 Deadline

The deadline is Unit 2's `fulfilment_due_at`, copied to the handover when it is created. A code entered at or after it is refused. Unit 2's fulfilment job then refunds the buyer in full (FR-PAY-08), and this unit cancels the handover when it receives `orders.escrow.refunded`. Because both the code entry and the job lock the order first (DC-14), an entry just before the deadline is either

recorded before the job runs, or refused after it; it can never be recorded for an order that has been refunded.

7 Courier Delivery

7.1 Addresses

Exact addresses are Confidential data (SRS Section 7). A saved address keeps only its label and area in clear. The street lines, the contact name and any landmark are encrypted by Unit 1's `IdentityCrypto` with AES-256-GCM, with the table, column and row id as associated data. Copying a ciphertext into another row therefore fails to decrypt, so a shipment takes a copy re-encrypted under its own row id. The rules are:

- each party sees only their own address (FR-DLV-07); the other party sees the district;
- the courier receives both addresses and the parties' verified mobile numbers, over TLS, at booking, and nothing else (FR-PRV-01 names couriers as recipients);
- staff see addresses masked; showing one in full needs a reason and is logged (SEC-ACC-05);
- a shipment's addresses are erased DV-30 after the order closes (30 days, or 30 days after a cancelled shipment is final), unless a dispute is open; the database refuses to erase them from a shipment that is not final.

7.2 Booking

On `orders.order.paid` for a courier order, the service creates the forward shipment in `preparing`, with the claimed quote, the fee, the buyer's drop-off address, and the CP-16 pickup deadline copied from the order. The booking worker then:

1. waits for the seller's pickup address if none is saved, and notifies the seller (the pickup deadline still runs from payment, and the listing form warns sellers about this);
2. calls `CourierPort.book` with the shipment id as the merchant reference, no cash to collect, and the billed weight and size;
3. on success, stores the consignment id and moves the shipment to `booked`;
4. on failure, retries with exponential backoff and jitter (DV-29: from 1 minute, doubling to at most 1 hour, 8 attempts), then opens a `booking_failed` issue.

While the provider is failing, a circuit breaker makes new courier quotes return `503 courier_unavailable`, so new courier checkouts are refused with a clear message. Existing shipments keep retrying (NFR-REL-04).

7.3 Shipment status

7.4 Applying courier status reports

Courier reports arrive by webhook, by the reconciliation query (Section 8), and, before a missed-pickup refund, by `syncPickup`. They can arrive twice, late or out of order. Each one is handled in four steps:

1. **Authenticate.** The webhook endpoint verifies the provider's signature with the current environment's secret and rejects a timestamp outside CP-32 (5 minutes) (SEC-PAY-03, SEC-PAY-04, IF-COM-03). The mock courier signs its webhooks the same way, so the same code path is tested.
2. **Confirm money-moving statuses.** `delivered` and `returned_to_seller` move money, so a webhook reporting them is treated as a hint. The service queries the provider's status API

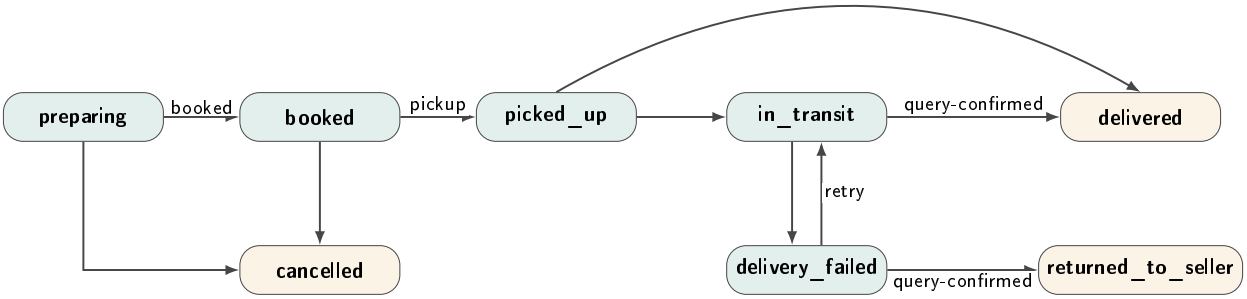


Figure 5: Shipment status (FR-DLV-10). Terminal states are shaded. Not drawn: `picked_up` and `in_transit` may also go straight to `delivery_failed` or `returned_to_seller`, and `delivery_failed` to `delivered`. The trigger `shipment_guard` refuses every other change.

and applies only what the query returns. This is the courier equivalent of FR-PAY-02. The database enforces it: these two statuses are accepted only with source `provider_query`.

3. **Apply once.** `delivery.apply_status` locks the shipment and records the provider's event id. A repeated id has no effect (`duplicate`). A report of an earlier stage, such as a pickup arriving after "in transit", changes nothing (`stale`). A report that skips `picked_up`, such as `delivered` straight after `booked`, implies the pickup at the same time. A report that makes no sense, such as progress on a cancelled booking, is refused and opens a `status_refused` issue. The provider's time is used, but never later than the server's clock and never earlier than the booking.
4. **Act on the result** (forward shipments), in the same DC-14 transaction:
   - a pickup by the deadline calls `recordFulfilmentStep(courier_pickup, at)`;
   - `delivered` calls `recordDeliveryEvent(courier_delivered, at)`; this is the delivery event (FR-DLV-10);
   - `returned_to_seller` calls `refundCourierReturned(charge)`, which refunds the buyer in full, including the fee, and books the courier's charge as a platform cost (FR-DLV-12).

A storm of 15 simultaneous reports (one event delivered 10 times, plus 5 other pickup events) produced exactly one applied pickup, 9 duplicates and 5 stale reports, and one history row.

7.5 Pickup deadline

If the courier has not picked up within CP-16 (3 days after payment), Unit 2's fulfilment job refunds the buyer in full (FR-DLV-11, FR-PAY-08). Webhooks can be delayed, so before refunding a courier order the job calls `DeliveryPort.syncPickup`. This queries the provider (with a DV-32 timeout of 10 seconds), outside any lock, and applies the result. A pickup that happened in time but was reported late is therefore counted. A pickup that happened after the deadline is recorded on the shipment but reported as late (`pickup_on_time` is false), so the escrow step is not recorded and the refund goes ahead. The parcel then follows the cancellation path below.

7.6 Cancellation

When an order is refunded before its delivery event (cancellation under FR-PAY-09, or a missed deadline), the service stores `order_cancelled_at` on the shipment and:

- cancels a `preparing` shipment directly;
- asks the provider to cancel a `booked` one, and applies the confirmed cancellation;
- if the courier already has the parcel, requests a return to the seller. When the return is reported, no second refund is made; the courier's charge is booked as a platform cost through `recordCourierCost` (I-21).

A pickup reported after the order was cancelled is recorded, flagged `after_cancel`, and raises a `pickup_after_cancel` issue, so that an operator makes sure the parcel goes back.

7.7 Failed delivery, return and non-delivery

- **Failed attempt.** `delivery_failed` counts the attempt and notifies both parties. The courier may try again (back to `in_transit`).
- **Return.** If the courier returns the item to the seller, the order is cancelled and the buyer refunded in full, including the delivery fee (FR-DLV-12). The SRS refunds the buyer in full even when the buyer refused the parcel. Repeated returns for one buyer are visible to Trust & Safety (Unit 7) in the fulfilment records, because each costs the platform the courier's charge.
- **Not received.** Once CP-17 (7 days) has passed since pickup without delivery, `DeliveryQueries.notReceivedAllowed` becomes true and the buyer may open a `not_received` dispute (FR-DLV-11, FR-DSP-01). Unit 2 raises its stalled-delivery review at CP-17 + CP-18 if nobody does.

7.8 Return shipments

When a dispute is decided with "refund after return" (FR-DSP-06(d)), Unit 7 calls `quoteReturn` and `bookReturn`. The return is a second shipment for the same order, in the `return` direction, from the buyer to the seller, priced from the same fare table. The decision records who pays the return fare. When the return is delivered, or the seller confirms receipt, `delivery.return.delivered` tells Unit 7 to complete the refund.

7.9 The mock courier

Pathao and Steadfast sandbox access is not yet confirmed, so the mock courier is the default (C-3, SRS open issue). It is a module inside the backend that implements the provider side of `CourierPort`:

- a booking API that returns consignment ids like `MOCK-` followed by random characters, and honours the merchant reference, so it is idempotent;
- a status API;
- signed webhooks sent to the normal webhook endpoint;
- a simulator page for demonstrations, where an operator moves a parcel through pickup, transit, a failed attempt, delivery or return, with an optional return charge. Optionally, parcels advance automatically after configurable delays.

Every mock shipment is `simulated` (enforced by a database constraint). The order view, tracking page, notifications and documentation all state that the courier booking is simulated (FR-DLV-14, NFR-CMP-05). The module and its endpoints exist only in sandbox builds.

7.10 Operator issues

Issue	Raised when, and what the operator does
`code_locked`	CP-45 wrong code entries. The operator contacts both parties, then unlocks or lets the deadline refund.
`booking_failed`	The booking still fails after DV-29. The operator books manually with the provider, or cancels the order (full refund).
`pickup_after_cancel`	The courier took a parcel for a refunded order. The operator ensures the return.

Issue	Raised when, and what the operator does
`status_refused`	A courier report the rules cannot place. The operator checks with the provider.
`return_failed`	A return shipment fails. The operator arranges the return with both parties.

8 Scheduled Jobs

Each job scans only due rows through a partial index, and is safe to run on several instances at once (`FOR UPDATE SKIP LOCKED`), as in Units 2 and 3 (NFR-REL-03).

Job	Every	Action
Booking	1 min	Book `preparing` shipments that are due, with backoff (DV-29).
Status reconciliation	30 min	Query the provider for shipments in flight with no report for 6 hours, and apply the answer (DV-31). This catches lost webhooks, as FR-PAY-15 does for payments.
Proposal lapse	15 min	Mark open proposals whose time has passed as `lapsed`.
Address erasure	daily	Erase shipment addresses after DV-30.
Provider events	daily	Delete event ids older than CP-32 (30 days).

Deadline reminders for the meetup handover and the courier pickup (FR-NTF-07) are sent by Unit 6 from the deadlines this unit publishes; the refund at the deadline is Unit 2's job.

9 Security and Privacy

- **Authorisation.** Each endpoint names its party: buyer, seller or "the other party" for proposals; the seller alone enters the code; the buyer alone sees it. The database functions check the same parties again, so a service bug cannot let a third person propose, answer or enter a code (tested). Staff actions are limited to those listed in the API, each with a reason, and audited (SEC-ACC-01 to SEC-ACC-03).
- **The handover code** is derived, never stored, never logged, and returned only to the buyer. Entry is limited to CP-45 wrong attempts, then locked; it is also rate-limited (Section 6.4).
- **Location privacy.** Parties see each other's district only. Meetup suggestions use snapped locations, and the midpoint is never returned. Distances are whole kilometres from rounded coordinates (FR-DLV-07).
- **Addresses** are encrypted field by field, shown only to their owner and the courier, masked for staff, and erased on schedule (Section 7.1).
- **Courier integration.** Webhooks are signed, time-limited and idempotent. Money-moving statuses are confirmed by a status query. Booking is idempotent. Credentials are separate per environment, and provider endpoints are fixed (SEC-PAY-03 to SEC-PAY-05, SEC-PAY-07, SEC-INP-05).
- **Integrity.** Code entries, shipment history and fare tables are append-only, and the application role cannot update them or delete shipments (tested). Quotes cannot be changed once issued.
- **Abuse limits.** Quotes are limited to 30 per hour per user (DV-23), and code entry to 10 per minute per seller (DV-28) (SEC-ABU-01).

9.1 Threats considered

Threat	Defence
Seller records a handover without meeting	Needs the buyer's code: at most 5 guesses at 6 digits, then locked.
Buyer takes the item without giving the code	The seller is told to hand over only after "Handover recorded" appears.
Buyer pressed to give the code before inspecting	The instruction says what the code means; a dispute remains possible within CP-18 for `not_as_described` or `damaged`.
Location stalking through meetup suggestions	Snapped locations; no midpoint in the API; district only.
Forged or replayed courier webhook	Signature, timestamp tolerance, event-id store; delivery and return confirmed by query.
Courier collects cash at the door	Bookings always set no cash to collect; the order view says the item is paid for.
Fee manipulation	The fee comes only from a server-issued quote, fixed and claimed once; client amounts are ignored (SEC-PAY-06).

10 Parameters

SRS parameters used: CP-14 (self-pickup threshold); CP-15 and CP-16 (handover and pickup deadlines, read from the order); CP-17 (non-delivery dispute); CP-26 (delivery margin, applied by Unit 2); CP-32 (webhook tolerance and event retention); CP-35 (rate limits); CP-43 (fare table); CP-45 (wrong code entries).

Design values introduced (continuing DV-1 to DV-22):

ID	Value	Default	Reason
DV-23	Quote validity; quote rate limit	30 minutes; 30 quotes per hour per user	Longer than the CP-13 checkout hold.
DV-24	Largest billed weight by courier	10 kg	Rate cards cap parcel weight; lower it if the provider's limit is lower (Paperfly's is 8 kg).
DV-25	Minimum billed weight by size class	small 0, medium 2 kg, large 5 kg; oversize not by courier	Couriers charge bulky parcels by volume.
DV-26	Meetup time window	at least 1 h ahead, at least 2 h before the deadline	No meeting the deadline cuts short.
DV-27	Meetup suggestions	3 points; locations snapped to 0.02°	Choice without revealing locations.
DV-28	Handover code	6 digits, HMAC-SHA-256 derivation; entry 10 per minute per seller	As OTPs (SEC-AUTH-09); never stored.
DV-29	Booking retries	from 1 min, doubling to 1 h; 8 attempts	Then an operator issue.
DV-30	Shipment address retention	30 days after the order closes, unless a dispute is open	Long enough for a late dispute; then erased.
DV-31	Courier status reconciliation	every 30 min, shipments silent for 6 h	Catches lost webhooks.
DV-32	Pickup query before a missed-pickup refund	10-second timeout	A late webhook does not cost the seller the sale.

11 Testing

Level	Cases
Database (done)	93 checks in `04_delivery_tests.sql`, each rejection required to fail on its intended rule. They cover zones, billed weight, fares and fare versions; the 2 km recommendation boundary; quotes, addresses, claiming and expiry; meetup suggestions and snapping; proposals, counter-proposals, rescheduling and lapse; the handover code, the lock and the unlock; the deadline; shipments, their quote and addresses; duplicates, stale and implied statuses, query confirmation, late pickup, clock clamping, return charge, cancellation and refusals; non-delivery eligibility; return shipments; issues; and grants. The Unit 1 to 3 scripts still pass with this schema loaded.
Concurrency (done)	Five races, three runs each on fresh databases, with the same counts every time: 20 wrong codes (4 wrong, 1 lock, 15 refused); 10 correct codes (1 accepted); 20 proposals (1 open); 15 courier reports (1 applied); 10 claims of one quote (1 claimed). See Appendix B.1.
Unit	Code derivation (fixed vectors, 6 digits, leading zeros); constant-time comparison; snapping; the webhook signature and timestamp checks; the circuit breaker; the indicative fee equals the quoted fee.
Integration	Handover plus escrow is atomic (an escrow error rolls back the code entry). A deadline race between a code entry and Unit 2's job. Mock courier end to end: book, pickup, transit, failed attempt, delivered, with the escrow's delivery event; return to seller with a charge; cancellation before and after pickup. `syncPickup` counts a late-reported pickup.
Security	Negative authorisation tests for every endpoint (SEC-SDL-09). The seller's responses never contain the code. Logs contain no code or address (checked by a log scan in CI).

12 Notes for Integration

I-19 Unit 2: `orders.order.paid` payload. It must carry the fulfilment method, buyer, seller, `fulfilment_due_at` and the delivery quote id.

I-20 Unit 2: `OrdersPort.lockForFulfilment(orderId, tx)`. New. Locks the order row and returns its status, fulfilment, parties, deadline and whether the step or delivery event is recorded (DC-14).

I-21 Unit 2: quotes and courier costs. `DeliveryPort.getQuote` becomes `claimQuote`, which also checks quantity and allows one use. New `EscrowCommands.recordCourierCost(orderId, amount, tx)` books a return charge for an order that was already refunded.

I-22 Unit 2: fulfilment job. Before refunding a courier order for a missed pickup, it calls `DeliveryPort.syncPickup` outside its lock. `recordFulfilmentStep` is called only for a pickup made by the deadline.

I-23 Unit 3. New `ListingsQueries.fulfilmentFacts(listingId)`. The listing form, and a constraint added to `listings.listings`, should refuse courier options for the `over_10kg` weight class and the `oversize` size class, which no fare covers (DV-24, DV-25).

I-24 Unit 1. New `IdentityQuery.getApproxArea(userId)`. Add `HANDOVER_KEY` and the courier API and webhook secrets to the key table (Unit 1 Section 3).

I-25 Unit 6 delivers the meetup, handover, lock and shipment notifications, and the deadline reminders (FR-NTF-07). No notification may contain the handover code.

I-26 Unit 7 hosts the delivery issue queue and the unlock action. It uses `fulfilmentRecord` in the case file (FR-DSP-05), `notReceivedAllowed` for `not_received` disputes, and `quoteReturn` and `bookReturn` for outcome (d).

I-27 Unit 8 (frontend) shows the code screen with its instruction, the seller's entry screen with the

"hand over only after" instruction, a "simulated" banner on every mock shipment, and map links for meetup points only.

I-28 SRS clarifications proposed:

- FR-DLV-08: distance enters the fare through the zone, as in the cited rate cards;
- FR-DLV-10: add the internal statuses `preparing` and `cancelled`, and state that `delivered` is confirmed by a status query;
- FR-DLV-11: a pickup made after CP-16 does not count, and the parcel is returned;
- FR-LST-03 refers to FR-DLV-07 for the fare model; it should refer to FR-DLV-08;
- CP-43: name the rate card used for the seed values, and add DV-23 to DV-32 to the SRS parameter table.

A Traceability

Requirement	Design element	Section
FR-DLV-01	Distance from rounded coordinates, shown in whole km	5
FR-DLV-02	`recommended_method`; buyer chooses among allowed methods; no fee for self-pickup	5
FR-DLV-03	Curated `meetup_points`; `suggest_points` near the midpoint	6
FR-DLV-04	`propose_meetup`, `respond_meetup`; one open proposal; agreed point on the handover	6
FR-DLV-05	Handover only for paid orders; no cash option; courier bookings collect no cash	6, 3.3
FR-DLV-06	Derived code shown to the buyer only; `record_code_entry`; CP-45 lock; unlock	6.4
FR-DLV-07	District only; snapped midpoint; encrypted addresses, own address only	6, 7.1
FR-DLV-08	Zones, billed weight, versioned fare table; quote fixed at checkout	5
FR-DLV-09	One total; no courier choice for the buyer	5
FR-DLV-10	Shipment status machine; query-confirmed delivery as the delivery event	7.4
FR-DLV-11	Pickup deadline with `syncPickup`; `not_received_allowed` after CP-17	7.4, 7.6
FR-DLV-12	Return to seller: full refund including fee; return charge as platform cost	7.4, 7.6
FR-DLV-13	Fee fixed on the order; split by Unit 2	5
FR-DLV-14	`simulated` on every mock shipment (constraint); labels	7.9
FR-PAY-07	Delivery event recorded only by code or query-confirmed delivery	6.4, 7.4
FR-PAY-08	Fulfilment step via DC-14; entry refused after the deadline	2.3, 6.4
FR-PAY-09	Handover and shipment cancelled on refund before delivery	7.6
FR-DSP-01	`notReceivedAllowed`	7.6
FR-DSP-05	`fulfilmentRecord`: proposals, code entries, shipments, history	3.1
FR-DSP-06	Return shipments; `delivery.return.delivered`	7.6
FR-NTF-07	Deadlines and events for reminders	8
FR-PRV-01	Courier named as recipient of addresses and phone numbers	7.1
SEC-PAY-03	Webhook signature check	7.4
SEC-PAY-04	Timestamp tolerance; `provider_events`	7.4
SEC-PAY-05	Separate courier credentials per environment	3.3
SEC-PAY-06	Fee only from a server-issued quote	5
SEC-PAY-07	Shipment id as idempotent merchant reference	3.3
SEC-DAT-07	Code and addresses never logged	6.4
SEC-ACC-05	Masked addresses for staff; logged display	7.1
SEC-INP-05	Fixed provider endpoints	3.3
SEC-ABU-01	Quote and code-entry limits	6.4
IF-SW-04	`CourierPort`; mock, Pathao and Steadfast adapters	3.3
IF-COM-03	Dedicated webhook endpoint	3.1

Requirement	Design element	Section
NFR-REL-03	Jobs on partial indexes, SKIP LOCKED	8
NFR-REL-04	Circuit breaker refuses new courier checkouts	7.2
NFR-CMP-05	Simulated labels	7.9
DR-06	Proposals, code entries, shipments, shipment history	4

B Database Schema and Tests

B.1 Test results

Loaded after the Unit 1 to Unit 3 schemas in PostgreSQL 16, using PostgreSQL 15 features only. Every check passed. The concurrency results follow the script output.

```
PASS  CP-43 both ends inside Dhaka: inside zone
PASS  CP-43 inside Dhaka to a suburb: suburbs zone
PASS  CP-43 suburb to outside Dhaka: outside zone
PASS  a listing near Dhanmondi is inside Dhaka; near Savar (same district) is a suburb
PASS  a district with no listed areas is outside Dhaka
PASS  DV-25 billed weight: 1 kg small = 1; 1 kg large = 5 (volumetric); 3 x 2 kg = 6
PASS  DV-24 over 10 kg in total, an over-10 kg item or an oversize item cannot go by courier
PASS  fare v1: inside Dhaka 1 kg = BDT 50; 5 kg = 65 + 3 x 15 = BDT 110; outside 2 kg = BDT 120
PASS  a fare version is not used until it has all three zones
PASS  a complete newer version takes effect
PASS  fare rows are append-only (publish a new version instead) [append-only]
PASS  a fare must not fall with weight [fare_rises_with_weight]
PASS  a published fare version names the administrator [fare_version_has_creator]
PASS  FR-DLV-02 at or below CP-14 (2 km) self-pickup is recommended; above it, courier
PASS  FR-DLV-02 only allowed methods are recommended
PASS  FR-DLV-08 Dhanmondi to Mirpur, 1 kg: inside Dhaka, BDT 50, distance recorded
PASS  FR-DLV-08 Dhanmondi to Savar: suburbs, BDT 80
PASS  FR-DLV-08 Dhanmondi to Chattogram, 2 units: outside Dhaka, 2 kg, BDT 120
PASS  FR-DLV-02 no courier quote for a pickup-only listing [does not offer courier]
PASS  DV-24 no quote for a parcel over the courier limit [too heavy or too large]
PASS  FR-DLV-07 a quote uses the buyer's own drop-off address [buyer's own]
PASS  the drop-off address must lie in the quoted area [in the quoted area]
PASS  a checkout quote always has a drop-off address [checkout_quote_has_address]
PASS  FR-DLV-08 a quote is fixed once issued [fixed once issued]
PASS  a quote is claimed only by the buyer it was issued to [does not match]
PASS  a quote is claimed only for the quantity it priced [does not match]
PASS  FR-DLV-08 claiming fixes the fee on the order (BDT 50)
PASS  a quote is used by one order only [already used]
PASS  DV-23 an expired quote cannot be used [quote expired]
PASS  FR-DLV-03 suggestions are the nearest active points to the midpoint, in order
PASS  FR-DLV-07 the midpoint uses snapped locations: moving one party by 300 m does not change it
PASS  a handover starts as arranging [starts as arranging]
PASS  FR-DLV-04 only the buyer or the seller can propose a meetup [only the buyer or the seller]
PASS  DV-26 a meetup must be at least the lead time ahead [meetup time must be]
PASS  DV-26 a meetup must leave the buffer before the CP-15 deadline [meetup time must be]
PASS  FR-DLV-03 a meetup is only at an active curated point [not an active meetup point]
PASS  FR-DLV-04 a counter-proposal replaces the open one; only one is open
PASS  FR-DLV-04 the proposer cannot accept their own proposal [only the other party]
PASS  FR-DLV-04 the other party accepts; the agreed point and time are recorded
PASS  a proposal cannot be edited after it is made [cannot be edited]
PASS  rescheduling: the earlier agreement stands until the new time is accepted
PASS  rescheduling: accepting the new time supersedes the old agreement
PASS  an answered proposal cannot be answered again [no longer open]
PASS  a proposal whose time has passed lapses instead of being accepted
PASS  FR-DLV-06 only the seller enters the handover code [only the seller]
PASS  CP-45 must stay within its permitted range 3 to 10 [between 3 and 10]
PASS  FR-DLV-06 wrong entries 1 to 4 are counted
PASS  FR-DLV-06 the 5th wrong entry (CP-45) locks code entry, opens an issue and notifies both
PASS  FR-DLV-06 no entry is accepted while locked, not even the right code [code entry is locked]
PASS  the wrong-entry count cannot be reset by an ordinary update [change only by an unlock]
PASS  an operator unlock issues code version 2, resets the count and keeps the agreement
PASS  FR-DLV-06 the correct code records the handover (the delivery event)
PASS  after the handover no further code is accepted [handover is closed]
PASS  a completed handover cannot be reopened or cancelled [handover is closed]
PASS  DR-06 every entry is logged with its code version (5 wrong on v1, 1 right on v2)
```

```
PASS  the code-entry log is append-only [append-only]
PASS  FR-PAY-08 no handover is recorded after the CP-15 deadline [deadline has passed]
PASS  an order cancelled before handover cancels the handover
PASS  a shipment uses its own claimed quote and fee [own claimed quote and fee]
PASS  FR-DLV-14 a mock booking is always marked simulated [shipment_simulated_iff_mock]
PASS  one forward shipment per order [shipment_one_per_direction]
PASS  FR-DLV-07 a booking needs the seller's pickup address [shipment_addresses]
PASS  a booking needs the provider's consignment id [shipment_booked_fields]
PASS  the consignment id is fixed once booked [consignment id is fixed]
PASS  FR-DLV-10 pickup by webhook is applied, on time, and reported for the fulfilment step
PASS  SEC-PAY-04 the same provider event again has no effect
PASS  FR-DLV-10 in transit is applied
PASS  a late-arriving older status (picked up after in transit) is ignored as stale
PASS  FR-DLV-10 a webhook alone cannot record delivery [provider status query]
PASS  FR-DLV-10 delivery confirmed by status query is the delivery event
PASS  nothing changes a delivered shipment (a later failure report is stale)
PASS  a final shipment cannot be edited [shipment is final]
PASS  DR-06 the history records every status in order, with its source
PASS  DV-30 a final shipment's addresses can be erased
PASS  a delivery reported straight after booking implies the pickup, at the same time
PASS  FR-DLV-11 a pickup after the CP-16 deadline is recorded but reported as late
PASS  a provider time in the future is clamped to the server time
PASS  FR-DLV-12 return to seller (confirmed by query) records the courier's charge
PASS  a pickup after the order was cancelled is flagged and raises an operator issue
PASS  a provider-confirmed cancellation before pickup cancels the shipment
PASS  progress reported on a cancelled booking is refused and sent to an operator
PASS  a status that skips the booking (preparing to picked up) is refused
PASS  the courier's return charge is recorded only on a return [shipment_charge_only_on_return]
PASS  DV-30 addresses are erased only after the shipment is final [shipment_erase_only_when_final]
PASS  FR-DLV-11 a not_received dispute is allowed CP-17 (7 days) after pickup without delivery
PASS  FR-DLV-11 ... but not before
PASS  FR-DSP-06(d) a return quote is claimed for an order that already used a checkout quote
PASS  FR-DSP-06(d) the return's delivery is published for Disputes to complete the refund
PASS  an issue is resolved only with a staff member and a reason [issue_resolution]
PASS  DC-6 outbox events are named delivery.<entity>.<past> [outbox_event_type_check]
PASS  application role cannot rewrite shipment history [permission denied]
PASS  application role cannot delete shipments [permission denied]
PASS  SEC-PAY-04 the application purges event ids older than CP-32 (30 days), and only those

93 checks passed (04_delivery_tests.sql, PostgreSQL 16.15 with the Unit 1-3 schemas loaded). The
Unit 1-3 test scripts still pass with Unit 4 loaded (46, 45 and 67 checks).

Concurrency (04_delivery_concurrency.sh; sessions released together by an advisory lock).
Three runs on fresh databases gave the same counts; only the quote race's winner differed.
  Wrong-code race: 20 sessions enter a wrong code at once (CP-45 = 5)
    results: wrong 4, locked 1, refused as locked 15
    database: wrong_entries=5, status=locked, logged entries=5
  Correct-code race: 10 sessions enter the right code at once
    results: accepted 1, refused as closed 9
    database: completed events=1
  Proposal race: 20 sessions (buyer and seller alternately) propose at once
    results: created 20, errors 0
    database: open=1, superseded=19
  Webhook storm: the same pickup event delivered 10 times plus 5 other pickup events, at once
    results: applied 1, duplicate 9, stale 5
    database: picked_up history rows=1
  Quote race: 10 orders claim the same quote at once
    results: claimed 1, refused as used 9
    database: quote order=d0000000-0000-4000-8000-0000000000cd
```

B.2 Schema

```
-- ================================================================
-- TrustBazaar - Unit 4: Delivery & Handover - PostgreSQL 15 schema
-- Design document: docs/design/04_delivery.pdf
-- Requirements: SRS v2.0 FR-DLV-01..14; FR-PAY-08 (fulfilment side); FR-DSP-05 and
--   FR-DSP-06(d) (delivery records, return shipments); SEC-PAY-03/04/07 (courier
--   webhooks); FR-PRV-01, NFR-CMP-05, DR-06.
-- Depends on: 01_identity_access.sql (identity.users, identity.staff_accounts,
--   identity.forbid_change), 02_orders_payments.sql (orders.orders),
--   03_listings_auctions.sql (listings.listings, listing enums, listings.distance_km).
-- Conventions DC-1..DC-14. Foreign keys only to earlier units (DC-5). No function here
-- reads another unit's tables (DC-1): order and listing facts arrive as arguments,
```

```
-- obtained through Unit 2 and Unit 3 ports.
-- ====================================================================

CREATE SCHEMA IF NOT EXISTS delivery;

-- ------------------------------------------------------------------ types
CREATE TYPE delivery.zone            AS ENUM ('inside_dhaka', 'dhaka_suburbs', 'outside_dhaka');   -- CP-43
CREATE TYPE delivery.point_kind      AS ENUM ('police_station', 'shopping_mall', 'bank_branch', 'other_public
    ');
CREATE TYPE delivery.handover_status AS ENUM ('arranging', 'agreed', 'completed', 'locked', 'cancelled');
CREATE TYPE delivery.proposal_status AS ENUM ('proposed', 'accepted', 'declined', 'withdrawn', 'superseded',
    'lapsed');
CREATE TYPE delivery.quote_purpose   AS ENUM ('checkout', 'return');
CREATE TYPE delivery.provider        AS ENUM ('mock', 'pathao', 'steadfast');                      -- IF-SW
    -04
CREATE TYPE delivery.direction       AS ENUM ('forward', 'return');                                -- return
    : FR-DSP-06(d)
CREATE TYPE delivery.shipment_status AS ENUM
  ('preparing',                                  -- waiting for the seller's pickup address or the booking
     call
   'booked', 'picked_up', 'in_transit', 'delivered', 'delivery_failed', 'returned_to_seller',  -- FR-DLV-10
   'cancelled');                                 -- booking withdrawn before pickup
CREATE TYPE delivery.status_source   AS ENUM ('provider_webhook', 'provider_query', 'system', 'operator');
CREATE TYPE delivery.address_kind    AS ENUM ('pickup', 'dropoff');
CREATE TYPE delivery.issue_kind      AS ENUM
  ('booking_failed', 'pickup_after_cancel', 'status_refused', 'code_locked', 'return_failed');
CREATE TYPE delivery.issue_status    AS ENUM ('open', 'resolved');

-- ====================================================================
-- Areas and zones (FR-DLV-08, CP-43). Curated reference data maintained by an
-- administrator. Buyers choose a drop-off area from this list; the zone of a
-- seller's listing location is that of the nearest area in the same district.
-- ====================================================================
CREATE TABLE delivery.areas (
  id        integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  district  varchar(40) NOT NULL,
  name_en   varchar(80) NOT NULL,
  name_bn   varchar(80) NOT NULL,
  zone      delivery.zone NOT NULL,
  lat       numeric(6,3) NOT NULL CHECK (lat BETWEEN 20.0 AND 27.0),     -- centroid, rounded (~110 m)
  lng       numeric(6,3) NOT NULL CHECK (lng BETWEEN 88.0 AND 93.0),
  active    boolean NOT NULL DEFAULT true,
  UNIQUE (district, name_en)
);
CREATE INDEX areas_district ON delivery.areas (lower(district)) WHERE active;

-- zone of the route: both ends inside Dhaka -> inside; both within Dhaka or its suburbs -> suburbs; else
    outside
CREATE FUNCTION delivery.route_zone(a delivery.zone, b delivery.zone) RETURNS delivery.zone
LANGUAGE sql IMMUTABLE AS $$
  SELECT CASE
    WHEN a = 'inside_dhaka' AND b = 'inside_dhaka' THEN 'inside_dhaka'::delivery.zone
    WHEN a <> 'outside_dhaka' AND b <> 'outside_dhaka' THEN 'dhaka_suburbs'::delivery.zone
    ELSE 'outside_dhaka'::delivery.zone END
$$;

-- zone of a point given only its district and rounded coordinates (a listing location)
CREATE FUNCTION delivery.zone_at(p_district varchar, p_lat numeric, p_lng numeric) RETURNS delivery.zone
LANGUAGE sql STABLE AS $$
  SELECT coalesce(
    (SELECT a.zone FROM delivery.areas a
      WHERE a.active AND lower(a.district) = lower(p_district)
      ORDER BY listings.distance_km(a.lat, a.lng, p_lat, p_lng), a.id LIMIT 1),
    'outside_dhaka'::delivery.zone)
$$;

-- ====================================================================
-- Fare table (FR-DLV-08, CP-43). Versioned and append-only; a quote records the version.
-- Fare for billed weight w kg: w <= 1 -> up_to_1kg; w <= 2 -> up_to_2kg;
-- else up_to_2kg + (w - 2) * per_extra_kg. Amounts in poisha.
-- ====================================================================
CREATE TABLE delivery.fare_tables (
  version         integer PRIMARY KEY CHECK (version >= 1),
  effective_from  timestamptz NOT NULL,
  source          text NOT NULL CHECK (length(btrim(source)) >= 10),   -- which published rate cards (R14)
```

```
  created_by      uuid REFERENCES identity.staff_accounts(id),         -- NULL only for the seeded version 1
  created_at      timestamptz NOT NULL DEFAULT now(),
  CONSTRAINT fare_version_has_creator CHECK (version = 1 OR created_by IS NOT NULL)
);
CREATE TABLE delivery.fare_rows (
  version       integer NOT NULL REFERENCES delivery.fare_tables(version),
  zone          delivery.zone NOT NULL,
  up_to_1kg     bigint NOT NULL CHECK (up_to_1kg > 0),
  up_to_2kg     bigint NOT NULL,
  per_extra_kg  bigint NOT NULL CHECK (per_extra_kg >= 0),
  PRIMARY KEY (version, zone),
  CONSTRAINT fare_rises_with_weight CHECK (up_to_2kg >= up_to_1kg)
);
CREATE TRIGGER fare_tables_append_only BEFORE UPDATE OR DELETE ON delivery.fare_tables
  FOR EACH ROW EXECUTE FUNCTION identity.forbid_change();
CREATE TRIGGER fare_rows_append_only BEFORE UPDATE OR DELETE ON delivery.fare_rows
  FOR EACH ROW EXECUTE FUNCTION identity.forbid_change();

-- the version in force at a time; a version is usable only once it has all three zones
CREATE FUNCTION delivery.fare_version_at(p_at timestamptz) RETURNS integer
LANGUAGE sql STABLE AS $$
  SELECT max(t.version) FROM delivery.fare_tables t
   WHERE t.effective_from <= p_at
     AND (SELECT count(*) FROM delivery.fare_rows r WHERE r.version = t.version) = 3
$$;

CREATE FUNCTION delivery.fare(p_version integer, p_zone delivery.zone, p_kg integer) RETURNS bigint
LANGUAGE sql STABLE AS $$
  SELECT CASE WHEN p_kg <= 1 THEN r.up_to_1kg
              WHEN p_kg <= 2 THEN r.up_to_2kg
              ELSE r.up_to_2kg + (p_kg - 2) * r.per_extra_kg END
    FROM delivery.fare_rows r WHERE r.version = p_version AND r.zone = p_zone
$$;

-- billed weight in whole kg: each unit counts as the larger of its weight-class bound
-- and the volumetric minimum of its size class (DV-25); NULL means "not by courier" (DV-24)
CREATE FUNCTION delivery.billed_kg(p_weight listings.weight_class, p_size listings.size_class,
                                   p_quantity integer, p_max_kg integer) RETURNS integer
LANGUAGE sql IMMUTABLE AS $$
  SELECT CASE WHEN w IS NULL OR s IS NULL OR greatest(w, s) * p_quantity > p_max_kg THEN NULL
              ELSE greatest(w, s) * p_quantity END
  FROM (SELECT CASE p_weight WHEN 'up_to_1kg' THEN 1 WHEN 'up_to_2kg' THEN 2 WHEN 'up_to_5kg' THEN 5
                             WHEN 'up_to_10kg' THEN 10 ELSE NULL END AS w,
               CASE p_size   WHEN 'small' THEN 0 WHEN 'medium' THEN 2 WHEN 'large' THEN 5
                             ELSE NULL END AS s) k
$$;

-- recommendation of FR-DLV-02: self-pickup at or below CP-14 if allowed, otherwise courier if allowed
CREATE FUNCTION delivery.recommended_method(p_options listings.fulfilment_options, p_distance_km numeric,
                                            p_threshold_km numeric) RETURNS text
LANGUAGE sql IMMUTABLE AS $$
  SELECT CASE p_options
    WHEN 'pickup_only'  THEN 'self_pickup'
    WHEN 'courier_only' THEN 'courier'
    ELSE CASE WHEN p_distance_km <= p_threshold_km THEN 'self_pickup' ELSE 'courier' END END
$$;

-- ====================================================================
-- Saved addresses (FR-DLV-07). Exact addresses are Confidential: the street lines and the
-- contact name are encrypted by the application with FIELD_KEY (AES-256-GCM, associated
-- data = table, column and row id, Unit 1 Section 3). Only the area is in clear.
-- ====================================================================
CREATE TABLE delivery.saved_addresses (
  id            uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id       uuid NOT NULL REFERENCES identity.users(id),
  kind          delivery.address_kind NOT NULL,
  label         varchar(40) NOT NULL,
  area_id       integer NOT NULL REFERENCES delivery.areas(id),
  address_enc   bytea CHECK (octet_length(address_enc) >= 30),     -- version byte, IV, ciphertext, tag
  created_at    timestamptz NOT NULL DEFAULT now(),
  deleted_at    timestamptz,
  CONSTRAINT saved_address_erased CHECK ((deleted_at IS NULL) = (address_enc IS NOT NULL))
);
CREATE INDEX saved_addresses_user ON delivery.saved_addresses (user_id, kind) WHERE deleted_at IS NULL;
```

```
-- ====================================================================
-- Quotes (FR-DLV-08, FR-DLV-09). Bound to buyer, listing and quantity; valid for DV-23;
-- claimed once, by one order (Unit 2 calls DeliveryPort.claimQuote at order creation).
-- ====================================================================
CREATE TABLE delivery.quotes (
  id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  purpose         delivery.quote_purpose NOT NULL,
  buyer_id        uuid NOT NULL REFERENCES identity.users(id),
  listing_id      uuid NOT NULL REFERENCES listings.listings(id),
  quantity        integer NOT NULL CHECK (quantity >= 1),
  fare_version    integer NOT NULL REFERENCES delivery.fare_tables(version),
  from_zone       delivery.zone NOT NULL,
  to_area_id      integer NOT NULL REFERENCES delivery.areas(id),
  dropoff_address_id uuid REFERENCES delivery.saved_addresses(id),  -- the buyer's address (checkout quotes)
  zone            delivery.zone NOT NULL,
  billed_kg       integer NOT NULL CHECK (billed_kg BETWEEN 1 AND 100),
  distance_km     numeric(6,1) NOT NULL CHECK (distance_km >= 0),
  fee             bigint NOT NULL CHECK (fee > 0),
  created_at      timestamptz NOT NULL DEFAULT now(),
  expires_at      timestamptz NOT NULL,
  order_id        uuid REFERENCES orders.orders(id),                -- set once, when claimed
  claimed_at      timestamptz,
  CONSTRAINT quote_one_per_order UNIQUE (order_id, purpose),
  CONSTRAINT quote_expiry_after_creation CHECK (expires_at > created_at),
  CONSTRAINT checkout_quote_has_address CHECK (purpose <> 'checkout' OR dropoff_address_id IS NOT NULL),
  CONSTRAINT quote_claim_fields CHECK ((order_id IS NULL) = (claimed_at IS NULL))
);
CREATE INDEX quotes_buyer ON delivery.quotes (buyer_id, created_at DESC);

CREATE FUNCTION delivery.quote_guard() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
  IF (NEW.id, NEW.purpose, NEW.buyer_id, NEW.listing_id, NEW.quantity, NEW.fare_version, NEW.zone,
      NEW.to_area_id, NEW.dropoff_address_id, NEW.billed_kg, NEW.fee, NEW.created_at, NEW.expires_at)
     IS DISTINCT FROM
     (OLD.id, OLD.purpose, OLD.buyer_id, OLD.listing_id, OLD.quantity, OLD.fare_version, OLD.zone,
      OLD.to_area_id, OLD.dropoff_address_id, OLD.billed_kg, OLD.fee, OLD.created_at, OLD.expires_at) THEN
    RAISE EXCEPTION 'a quote is fixed once issued' USING ERRCODE = 'check_violation';
  END IF;
  IF OLD.order_id IS NOT NULL AND NEW.order_id IS DISTINCT FROM OLD.order_id THEN
    RAISE EXCEPTION 'a quote is claimed only once' USING ERRCODE = 'check_violation';
  END IF;
  RETURN NEW;
END $$;
CREATE TRIGGER quote_guard BEFORE UPDATE ON delivery.quotes FOR EACH ROW EXECUTE FUNCTION delivery.
    quote_guard();

-- issue a quote. Listing facts (classes, district, rounded coordinates, fulfilment options)
-- come from Unit 3's ListingsQueries port; parameters from ConfigPort.
CREATE FUNCTION delivery.make_quote(
    p_purpose delivery.quote_purpose, p_buyer uuid, p_listing uuid, p_quantity integer,
    p_options listings.fulfilment_options, p_weight listings.weight_class, p_size listings.size_class,
    p_from_district varchar, p_from_lat numeric, p_from_lng numeric,
    p_to_area integer, p_validity interval, p_max_kg integer, p_address uuid DEFAULT NULL)
RETURNS delivery.quotes LANGUAGE plpgsql AS $$
DECLARE
  a delivery.areas; v integer; kg integer; z delivery.zone; fz delivery.zone; q delivery.quotes;
BEGIN
  IF p_purpose = 'checkout' AND p_options IS DISTINCT FROM 'courier_only' AND p_options IS DISTINCT FROM '
    pickup_or_courier' THEN
    RAISE EXCEPTION 'the seller does not offer courier for this listing' USING ERRCODE = 'check_violation';
  END IF;
  SELECT * INTO a FROM delivery.areas WHERE id = p_to_area AND active;
  IF NOT FOUND THEN RAISE EXCEPTION 'unknown drop-off area' USING ERRCODE = 'check_violation'; END IF;
  IF p_address IS NOT NULL AND NOT EXISTS (SELECT 1 FROM delivery.saved_addresses
       WHERE id = p_address AND user_id = p_buyer AND kind = 'dropoff' AND deleted_at IS NULL AND area_id =
    p_to_area) THEN
    RAISE EXCEPTION 'drop-off address must be the buyer''s own, in the quoted area' USING ERRCODE = '
    check_violation';
  END IF;
  kg := delivery.billed_kg(p_weight, p_size, p_quantity, p_max_kg);
  IF kg IS NULL THEN
    RAISE EXCEPTION 'parcel too heavy or too large for courier' USING ERRCODE = 'check_violation';
  END IF;
  v := delivery.fare_version_at(now());
  IF v IS NULL THEN RAISE EXCEPTION 'no fare table in force' USING ERRCODE = 'check_violation'; END IF;
```

```
  fz := delivery.zone_at(p_from_district, p_from_lat, p_from_lng);
  z  := delivery.route_zone(fz, a.zone);
  INSERT INTO delivery.quotes (purpose, buyer_id, listing_id, quantity, fare_version, from_zone, to_area_id,
                               dropoff_address_id, zone, billed_kg, distance_km, fee, expires_at)
  VALUES (p_purpose, p_buyer, p_listing, p_quantity, v, fz, a.id, p_address, z, kg,
          round(listings.distance_km(p_from_lat, p_from_lng, a.lat, a.lng)::numeric, 1),
          delivery.fare(v, z, kg), now() + p_validity)
  RETURNING * INTO q;
  RETURN q;
END $$;

-- claim at order creation, inside Unit 2's transaction (DC-13); returns the fee to store on the order
-- (a return quote is claimed the same way when Disputes books the return, with purpose 'return')
CREATE FUNCTION delivery.claim_quote(p_quote uuid, p_order uuid, p_buyer uuid, p_listing uuid, p_quantity
    integer,
                                     p_purpose delivery.quote_purpose DEFAULT 'checkout')
RETURNS bigint LANGUAGE plpgsql AS $$
DECLARE q delivery.quotes;
BEGIN
  SELECT * INTO q FROM delivery.quotes WHERE id = p_quote FOR UPDATE;
  IF NOT FOUND OR q.purpose <> p_purpose OR q.buyer_id <> p_buyer OR q.listing_id <> p_listing OR q.quantity
     <> p_quantity THEN
    RAISE EXCEPTION 'quote␣does␣not␣match␣this␣order' USING ERRCODE = 'check_violation';
  END IF;
  IF q.order_id IS NOT NULL THEN RAISE EXCEPTION 'quote␣already␣used' USING ERRCODE = 'check_violation'; END
     IF;
  IF now() >= q.expires_at THEN RAISE EXCEPTION 'quote␣expired' USING ERRCODE = 'check_violation'; END IF;
  UPDATE delivery.quotes SET order_id = p_order, claimed_at = now() WHERE id = p_quote;
  RETURN q.fee;
END $$;

-- ====================================================================
-- Safe meetup points (FR-DLV-03). Public places, so their exact location is shown.
-- ====================================================================
CREATE TABLE delivery.meetup_points (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  kind        delivery.point_kind NOT NULL,
  name_en     varchar(120) NOT NULL,
  name_bn     varchar(120) NOT NULL,
  district    varchar(40)  NOT NULL,
  lat         numeric(8,5) NOT NULL CHECK (lat BETWEEN 20.0 AND 27.0),
  lng         numeric(8,5) NOT NULL CHECK (lng BETWEEN 88.0 AND 93.0),
  guidance    varchar(300),                          -- e.g. "main gate, inside the guarded forecourt"
  active      boolean NOT NULL DEFAULT true,
  added_by    uuid NOT NULL REFERENCES identity.staff_accounts(id),
  created_at  timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX meetup_points_geo ON delivery.meetup_points (lat, lng) WHERE active;

-- nearest active points to the midpoint of the two parties' locations (DV-27). Each location is
-- first snapped to a coarse grid (p_grid degrees), because a party who knows their own location
-- and the midpoint could otherwise work out the other's (other = 2 x midpoint - own). The
-- arithmetic mean of coordinates is accurate to well under 1 km at meetup distances. The
-- distance column is for ordering only and is not returned by the API.
CREATE FUNCTION delivery.suggest_points(lat1 numeric, lng1 numeric, lat2 numeric, lng2 numeric, n integer,
    p_grid numeric)
RETURNS TABLE (point_id uuid, name_en varchar, kind delivery.point_kind, km_from_midpoint numeric)
LANGUAGE sql STABLE AS $$
  WITH m AS (SELECT (round(lat1 / p_grid) + round(lat2 / p_grid)) * p_grid / 2 AS lat,
                    (round(lng1 / p_grid) + round(lng2 / p_grid)) * p_grid / 2 AS lng)
  SELECT p.id, p.name_en, p.kind, round(listings.distance_km(p.lat, p.lng, m.lat, m.lng)::numeric, 1)
    FROM delivery.meetup_points p, m
   WHERE p.active
   ORDER BY listings.distance_km(p.lat, p.lng, m.lat, m.lng), p.id
   LIMIT n
$$;

-- ====================================================================
-- Meetup handover (FR-DLV-04 to FR-DLV-06). One row per self-pickup order, created when
-- Unit 2 reports the order paid. The handover code is never stored: it is derived as
-- HMAC-SHA-256(HANDOVER_KEY, order_id || ':' || code_version), reduced to 6 digits (DV-28).
-- ====================================================================
CREATE TABLE delivery.handovers (
  order_id            uuid PRIMARY KEY REFERENCES orders.orders(id),
  buyer_id            uuid NOT NULL REFERENCES identity.users(id),
```

```
  seller_id           uuid NOT NULL REFERENCES identity.users(id),
  status              delivery.handover_status NOT NULL DEFAULT 'arranging',
  due_at              timestamptz NOT NULL,                  -- the order's fulfilment deadline (CP-15)
  code_version        smallint NOT NULL DEFAULT 1 CHECK (code_version >= 1),
  wrong_entries       smallint NOT NULL DEFAULT 0 CHECK (wrong_entries BETWEEN 0 AND 10),
  agreed_proposal_id  uuid,                                  -- FK added below
  locked_at           timestamptz,
  completed_at        timestamptz,
  cancelled_at        timestamptz,
  created_at          timestamptz NOT NULL DEFAULT now(),
  CONSTRAINT handover_parties CHECK (buyer_id <> seller_id),
  CONSTRAINT handover_completed_fields CHECK ((status = 'completed') = (completed_at IS NOT NULL)),
  CONSTRAINT handover_locked_fields    CHECK ((status = 'locked')    = (locked_at IS NOT NULL)),
  CONSTRAINT handover_cancelled_fields CHECK ((status = 'cancelled') = (cancelled_at IS NOT NULL)),
  CONSTRAINT handover_agreed_has_proposal CHECK (status <> 'agreed' OR agreed_proposal_id IS NOT NULL)
);

CREATE FUNCTION delivery.handover_guard() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
  IF TG_OP = 'INSERT' THEN
    IF NEW.status <> 'arranging' OR NEW.wrong_entries <> 0 OR NEW.code_version <> 1 THEN
      RAISE EXCEPTION 'a␣handover␣starts␣as␣arranging' USING ERRCODE = 'check_violation';
    END IF;
    RETURN NEW;
  END IF;
  IF (NEW.order_id, NEW.buyer_id, NEW.seller_id, NEW.due_at) IS DISTINCT FROM (OLD.order_id, OLD.buyer_id,
     OLD.seller_id, OLD.due_at) THEN
    RAISE EXCEPTION 'handover␣parties␣and␣deadline␣are␣fixed' USING ERRCODE = 'check_violation';
  END IF;
  IF OLD.status IN ('completed', 'cancelled') THEN
    RAISE EXCEPTION 'handover␣is␣closed' USING ERRCODE = 'check_violation';
  END IF;
  IF NEW.status IS DISTINCT FROM OLD.status AND NOT ((OLD.status, NEW.status) IN (
       ('arranging'::delivery.handover_status, 'agreed'::delivery.handover_status),
       ('arranging', 'completed'), ('arranging', 'locked'), ('arranging', 'cancelled'),
       ('agreed', 'completed'), ('agreed', 'locked'), ('agreed', 'cancelled'),
       ('locked', 'arranging'), ('locked', 'agreed'), ('locked', 'cancelled'))) THEN
    RAISE EXCEPTION 'handover␣%␣->␣%␣is␣not␣permitted', OLD.status, NEW.status USING ERRCODE = '
     check_violation';
  END IF;
  -- the counter only rises, except when an operator resets a locked code, which issues a new code
  IF OLD.status = 'locked' AND NEW.status IN ('arranging', 'agreed') THEN
    IF NEW.code_version <> OLD.code_version + 1 OR NEW.wrong_entries <> 0 THEN
      RAISE EXCEPTION 'unlocking␣issues␣a␣new␣code␣and␣resets␣the␣counter' USING ERRCODE = 'check_violation';
    END IF;
  ELSIF NEW.code_version <> OLD.code_version OR NEW.wrong_entries < OLD.wrong_entries THEN
    RAISE EXCEPTION 'the␣code␣and␣its␣wrong-entry␣count␣change␣only␣by␣an␣unlock' USING ERRCODE = '
     check_violation';
  END IF;
  RETURN NEW;
END $$;
CREATE TRIGGER handover_guard BEFORE INSERT OR UPDATE ON delivery.handovers
  FOR EACH ROW EXECUTE FUNCTION delivery.handover_guard();

CREATE TABLE delivery.meetup_proposals (
  id            uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  order_id      uuid NOT NULL REFERENCES delivery.handovers(order_id),
  proposed_by   uuid NOT NULL REFERENCES identity.users(id),
  point_id      uuid NOT NULL REFERENCES delivery.meetup_points(id),
  meet_at       timestamptz NOT NULL,
  note          varchar(200),
  status        delivery.proposal_status NOT NULL DEFAULT 'proposed',
  created_at    timestamptz NOT NULL DEFAULT now(),
  responded_at  timestamptz,
  CONSTRAINT proposal_response_time CHECK ((status = 'proposed') = (responded_at IS NULL))
);
CREATE UNIQUE INDEX proposal_one_open ON delivery.meetup_proposals (order_id) WHERE status = 'proposed';
CREATE INDEX proposal_order ON delivery.meetup_proposals (order_id, created_at);
ALTER TABLE delivery.handovers ADD CONSTRAINT handover_agreed_fk
  FOREIGN KEY (agreed_proposal_id) REFERENCES delivery.meetup_proposals(id);

CREATE FUNCTION delivery.proposal_guard() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
  IF TG_OP = 'INSERT' THEN
```

```
    IF NEW.status <> 'proposed' THEN RAISE EXCEPTION 'a proposal starts as proposed' USING ERRCODE = '
     check_violation'; END IF;
    RETURN NEW;
  END IF;
  IF (NEW.order_id, NEW.proposed_by, NEW.point_id, NEW.meet_at, NEW.note, NEW.created_at)
     IS DISTINCT FROM (OLD.order_id, OLD.proposed_by, OLD.point_id, OLD.meet_at, OLD.note, OLD.created_at)
     THEN
    RAISE EXCEPTION 'a proposal cannot be edited; make a new one' USING ERRCODE = 'check_violation';
  END IF;
  IF NEW.status IS DISTINCT FROM OLD.status AND NOT ((OLD.status, NEW.status) IN (
       ('proposed'::delivery.proposal_status, 'accepted'::delivery.proposal_status),
       ('proposed', 'declined'), ('proposed', 'withdrawn'), ('proposed', 'superseded'), ('proposed', 'lapsed'
    ),
       ('accepted', 'superseded'))) THEN
    RAISE EXCEPTION 'proposal % -> % is not permitted', OLD.status, NEW.status USING ERRCODE = '
     check_violation';
  END IF;
  RETURN NEW;
END $$;
CREATE TRIGGER proposal_guard BEFORE INSERT OR UPDATE ON delivery.meetup_proposals
  FOR EACH ROW EXECUTE FUNCTION delivery.proposal_guard();

-- every code entry, right or wrong (DR-06); append-only
CREATE TABLE delivery.code_entries (
  id            bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  order_id      uuid NOT NULL REFERENCES delivery.handovers(order_id),
  seller_id     uuid NOT NULL REFERENCES identity.users(id),
  code_version  smallint NOT NULL,
  correct       boolean NOT NULL,
  at            timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX code_entries_order ON delivery.code_entries (order_id, id);
CREATE TRIGGER code_entries_append_only BEFORE UPDATE OR DELETE ON delivery.code_entries
  FOR EACH ROW EXECUTE FUNCTION identity.forbid_change();

-- ================================================================
-- Courier shipments (FR-DLV-10 to FR-DLV-14). One forward shipment per courier order and
-- at most one return shipment (dispute outcome (d), FR-DSP-06).
-- ================================================================
CREATE TABLE delivery.shipments (
  id                  uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  order_id            uuid NOT NULL REFERENCES orders.orders(id),
  direction           delivery.direction NOT NULL,
  provider            delivery.provider NOT NULL,
  simulated           boolean NOT NULL,                                   -- FR-DLV-14, NFR-CMP-05
  quote_id            uuid NOT NULL REFERENCES delivery.quotes(id),
  fee                 bigint NOT NULL CHECK (fee > 0),
  status              delivery.shipment_status NOT NULL DEFAULT 'preparing',
  pickup_due_at       timestamptz,                                        -- forward: the order's CP-16
     deadline
  consignment_id      varchar(64),
  pickup_area_id      integer REFERENCES delivery.areas(id),
  dropoff_area_id     integer NOT NULL REFERENCES delivery.areas(id),
  pickup_address_enc  bytea CHECK (octet_length(pickup_address_enc) >= 30),
  dropoff_address_enc bytea CHECK (octet_length(dropoff_address_enc) >= 30),
  addresses_erased_at timestamptz,                                        -- DV-30
  booking_attempts    smallint NOT NULL DEFAULT 0 CHECK (booking_attempts >= 0),
  next_booking_at     timestamptz,
  booked_at           timestamptz,
  picked_up_at        timestamptz,
  delivered_at        timestamptz,
  failed_attempts     smallint NOT NULL DEFAULT 0 CHECK (failed_attempts >= 0),
  returned_at         timestamptz,
  return_charge       bigint CHECK (return_charge >= 0),
  cancelled_at        timestamptz,
  order_cancelled_at  timestamptz,                                        -- set from Unit 2's cancellation
     event
  last_source         delivery.status_source NOT NULL DEFAULT 'system',
  last_event_id       varchar(100),
  created_at          timestamptz NOT NULL DEFAULT now(),
  CONSTRAINT shipment_one_per_direction UNIQUE (order_id, direction),
  CONSTRAINT shipment_simulated_iff_mock CHECK (simulated = (provider = 'mock')),
  CONSTRAINT shipment_forward_has_deadline CHECK ((direction = 'forward') = (pickup_due_at IS NOT NULL)),
  CONSTRAINT shipment_booked_fields CHECK (status IN ('preparing', 'cancelled')
         OR (consignment_id IS NOT NULL AND booked_at IS NOT NULL)),
```

```
  CONSTRAINT shipment_pickup_fields CHECK ((status IN ('picked_up', 'in_transit', 'delivered', '
     delivery_failed', 'returned_to_seller'))
         = (picked_up_at IS NOT NULL)),
  CONSTRAINT shipment_delivered_fields CHECK ((status = 'delivered') = (delivered_at IS NOT NULL)),
  CONSTRAINT shipment_returned_fields CHECK ((status = 'returned_to_seller') = (returned_at IS NOT NULL)),
  CONSTRAINT shipment_charge_only_on_return CHECK (return_charge IS NULL OR status = 'returned_to_seller'),
  CONSTRAINT shipment_cancelled_fields CHECK ((status = 'cancelled') = (cancelled_at IS NOT NULL)),
  CONSTRAINT shipment_time_order CHECK (picked_up_at IS NULL OR picked_up_at >= booked_at),
  -- once booked, the provider has both addresses; afterwards they are kept only until DV-30
  CONSTRAINT shipment_addresses CHECK (
         CASE WHEN addresses_erased_at IS NOT NULL THEN pickup_address_enc IS NULL AND dropoff_address_enc IS
      NULL
              WHEN status = 'preparing' THEN dropoff_address_enc IS NOT NULL
              ELSE pickup_address_enc IS NOT NULL AND dropoff_address_enc IS NOT NULL AND pickup_area_id IS
     NOT NULL END),
  CONSTRAINT shipment_erase_only_when_final CHECK (addresses_erased_at IS NULL
         OR status IN ('delivered', 'returned_to_seller', 'cancelled'))
);
CREATE UNIQUE INDEX shipments_consignment ON delivery.shipments (provider, consignment_id) WHERE
     consignment_id IS NOT NULL;
CREATE INDEX shipments_booking_due ON delivery.shipments (next_booking_at) WHERE status = 'preparing';
CREATE INDEX shipments_in_flight ON delivery.shipments (picked_up_at) WHERE status IN ('picked_up', '
     in_transit', 'delivery_failed');

CREATE FUNCTION delivery.shipment_guard() RETURNS trigger LANGUAGE plpgsql AS $$
DECLARE q delivery.quotes;
BEGIN
  IF TG_OP = 'INSERT' THEN
    IF NEW.status <> 'preparing' THEN RAISE EXCEPTION 'a␣shipment␣starts␣as␣preparing' USING ERRCODE = '
     check_violation'; END IF;
    SELECT * INTO q FROM delivery.quotes WHERE id = NEW.quote_id;
    IF q.fee <> NEW.fee OR q.order_id IS DISTINCT FROM NEW.order_id
       OR (q.purpose = 'checkout') <> (NEW.direction = 'forward') THEN
      RAISE EXCEPTION 'shipment␣must␣use␣its␣own␣claimed␣quote␣and␣fee' USING ERRCODE = 'check_violation';
    END IF;
    IF NEW.dropoff_area_id <> q.to_area_id THEN
      RAISE EXCEPTION 'drop-off␣area␣differs␣from␣the␣quoted␣area' USING ERRCODE = 'check_violation';
    END IF;
    RETURN NEW;
  END IF;
  IF (NEW.order_id, NEW.direction, NEW.provider, NEW.simulated, NEW.quote_id, NEW.fee, NEW.pickup_due_at, NEW
     .dropoff_area_id)
     IS DISTINCT FROM (OLD.order_id, OLD.direction, OLD.provider, OLD.simulated, OLD.quote_id, OLD.fee, OLD.
     pickup_due_at, OLD.dropoff_area_id) THEN
    RAISE EXCEPTION 'shipment␣identity,␣quote␣and␣fee␣are␣fixed' USING ERRCODE = 'check_violation';
  END IF;
  IF OLD.consignment_id IS NOT NULL AND NEW.consignment_id IS DISTINCT FROM OLD.consignment_id THEN
    RAISE EXCEPTION 'consignment␣id␣is␣fixed␣once␣booked' USING ERRCODE = 'check_violation';
  END IF;
  IF OLD.status <> 'preparing' AND (NEW.pickup_area_id IS DISTINCT FROM OLD.pickup_area_id
       OR (NEW.addresses_erased_at IS NULL AND (NEW.pickup_address_enc IS DISTINCT FROM OLD.
     pickup_address_enc
                                                OR NEW.dropoff_address_enc IS DISTINCT FROM OLD.
     dropoff_address_enc))) THEN
    RAISE EXCEPTION 'addresses␣are␣fixed␣once␣booked' USING ERRCODE = 'check_violation';
  END IF;
  IF OLD.status IN ('delivered', 'returned_to_seller', 'cancelled') AND NEW.status = OLD.status THEN
    -- a final shipment may only have its addresses erased
    IF (NEW.delivered_at, NEW.returned_at, NEW.return_charge, NEW.cancelled_at, NEW.picked_up_at, NEW.
     failed_attempts)
       IS DISTINCT FROM (OLD.delivered_at, OLD.returned_at, OLD.return_charge, OLD.cancelled_at, OLD.
     picked_up_at, OLD.failed_attempts) THEN
      RAISE EXCEPTION 'shipment␣is␣final' USING ERRCODE = 'check_violation';
    END IF;
    RETURN NEW;
  END IF;
  IF NEW.status IS DISTINCT FROM OLD.status AND NOT ((OLD.status, NEW.status) IN (
       ('preparing'::delivery.shipment_status, 'booked'::delivery.shipment_status), ('preparing', 'cancelled'
     ),
       ('booked', 'picked_up'), ('booked', 'cancelled'),
       ('picked_up', 'in_transit'), ('picked_up', 'delivered'), ('picked_up', 'delivery_failed'), ('picked_up
     ', 'returned_to_seller'),
       ('in_transit', 'delivered'), ('in_transit', 'delivery_failed'), ('in_transit', 'returned_to_seller'),
       ('delivery_failed', 'in_transit'), ('delivery_failed', 'delivered'), ('delivery_failed', '
     returned_to_seller'))) THEN
```

```
    RAISE EXCEPTION 'shipment␣%␣->␣%␣is␣not␣permitted', OLD.status, NEW.status USING ERRCODE = '
     check_violation';
  END IF;
  -- delivery and return move money: they are accepted only after a status query to the provider
  IF NEW.status IS DISTINCT FROM OLD.status AND NEW.status IN ('delivered', 'returned_to_seller')
     AND NEW.last_source NOT IN ('provider_query') THEN
    RAISE EXCEPTION 'delivered␣and␣returned␣need␣confirmation␣by␣a␣provider␣status␣query' USING ERRCODE = '
     check_violation';
  END IF;
  RETURN NEW;
END $$;
CREATE TRIGGER shipment_guard BEFORE INSERT OR UPDATE ON delivery.shipments
  FOR EACH ROW EXECUTE FUNCTION delivery.shipment_guard();

CREATE TABLE delivery.shipment_history (                                        -- DR-06, append-only
  id            bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  shipment_id   uuid NOT NULL REFERENCES delivery.shipments(id),
  from_status   delivery.shipment_status,
  to_status     delivery.shipment_status NOT NULL,
  source        delivery.status_source NOT NULL,
  event_id      varchar(100),
  at            timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX shipment_history_shipment ON delivery.shipment_history (shipment_id, id);
CREATE TRIGGER shipment_history_append_only BEFORE UPDATE OR DELETE ON delivery.shipment_history
  FOR EACH ROW EXECUTE FUNCTION identity.forbid_change();

CREATE FUNCTION delivery.shipment_log() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
  IF TG_OP = 'INSERT' OR NEW.status IS DISTINCT FROM OLD.status THEN
    INSERT INTO delivery.shipment_history (shipment_id, from_status, to_status, source, event_id)
    VALUES (NEW.id, CASE WHEN TG_OP = 'UPDATE' THEN OLD.status END, NEW.status, NEW.last_source, NEW.
     last_event_id);
  END IF;
  RETURN NULL;
END $$;
CREATE TRIGGER shipment_log AFTER INSERT OR UPDATE ON delivery.shipments
  FOR EACH ROW EXECUTE FUNCTION delivery.shipment_log();

-- courier webhook and query events, for idempotency (SEC-PAY-04); kept for CP-32, then deleted
CREATE TABLE delivery.provider_events (
  provider        delivery.provider NOT NULL,
  event_id        varchar(100) NOT NULL,
  consignment_id  varchar(64),
  reported_status delivery.shipment_status,
  occurred_at     timestamptz,
  outcome         varchar(20),
  received_at     timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (provider, event_id)
);
CREATE INDEX provider_events_age ON delivery.provider_events (received_at);

CREATE TABLE delivery.issues (                                                  -- operator queue (Unit 7)
  id            uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  order_id      uuid NOT NULL REFERENCES orders.orders(id),
  kind          delivery.issue_kind NOT NULL,
  detail        jsonb NOT NULL DEFAULT '{}',
  status        delivery.issue_status NOT NULL DEFAULT 'open',
  created_at    timestamptz NOT NULL DEFAULT now(),
  resolved_by   uuid REFERENCES identity.staff_accounts(id),
  resolved_at   timestamptz,
  resolution    text,
  CONSTRAINT issue_resolution CHECK ((status = 'resolved') = (resolved_at IS NOT NULL AND resolved_by IS NOT
     NULL
                                    AND length(btrim(coalesce(resolution, ''))) >= 10))
);
CREATE INDEX issues_open ON delivery.issues (created_at) WHERE status = 'open';
CREATE UNIQUE INDEX issues_one_open_per_kind ON delivery.issues (order_id, kind) WHERE status = 'open';

CREATE TABLE delivery.outbox (                                                  -- DC-6
  id              bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  event_type      varchar(80) NOT NULL CHECK (event_type ~ '^delivery\.[a-z_]+\.[a-z_]+$'),
  aggregate_id    uuid NOT NULL,
  payload         jsonb NOT NULL,
  created_at      timestamptz NOT NULL DEFAULT now(),
```

```
  attempts        smallint NOT NULL DEFAULT 0,
  next_attempt_at timestamptz NOT NULL DEFAULT now(),
  dispatched_at   timestamptz
);
CREATE INDEX delivery_outbox_pending ON delivery.outbox (next_attempt_at) WHERE dispatched_at IS NULL;

-- ====================================================================
-- Operations. Each locks the delivery row it changes; the caller has already locked the
-- order through OrdersPort.lockForFulfilment (lock order: listing, order, delivery row,
-- escrow, attempt - DC-14).
-- ====================================================================

-- FR-DLV-04: propose a meetup point and time. A new proposal replaces an open one.
CREATE FUNCTION delivery.propose_meetup(p_order uuid, p_user uuid, p_point uuid, p_meet_at timestamptz,
                                        p_note varchar, p_lead interval, p_buffer interval)
RETURNS uuid LANGUAGE plpgsql AS $$
DECLARE h delivery.handovers; pid uuid;
BEGIN
  SELECT * INTO h FROM delivery.handovers WHERE order_id = p_order FOR UPDATE;
  IF NOT FOUND THEN RAISE EXCEPTION 'no␣handover␣for␣this␣order' USING ERRCODE = 'check_violation'; END IF;
  IF p_user NOT IN (h.buyer_id, h.seller_id) THEN
    RAISE EXCEPTION 'only␣the␣buyer␣or␣the␣seller␣can␣propose' USING ERRCODE = 'insufficient_privilege';
  END IF;
  IF h.status NOT IN ('arranging', 'agreed') THEN
    RAISE EXCEPTION 'a␣meetup␣cannot␣be␣arranged␣now␣(%)', h.status USING ERRCODE = 'check_violation';
  END IF;
  IF NOT EXISTS (SELECT 1 FROM delivery.meetup_points WHERE id = p_point AND active) THEN
    RAISE EXCEPTION 'not␣an␣active␣meetup␣point' USING ERRCODE = 'check_violation';
  END IF;
  IF p_meet_at < now() + p_lead OR p_meet_at > h.due_at - p_buffer THEN
    RAISE EXCEPTION 'meetup␣time␣must␣be␣between␣%␣from␣now␣and␣%␣before␣the␣handover␣deadline', p_lead,
     p_buffer
      USING ERRCODE = 'check_violation';
  END IF;
  UPDATE delivery.meetup_proposals SET status = 'superseded', responded_at = now()
   WHERE order_id = p_order AND status = 'proposed';
  INSERT INTO delivery.meetup_proposals (order_id, proposed_by, point_id, meet_at, note)
  VALUES (p_order, p_user, p_point, p_meet_at, p_note) RETURNING id INTO pid;
  INSERT INTO delivery.outbox (event_type, aggregate_id, payload)
  VALUES ('delivery.meetup.proposed', p_order, jsonb_build_object('proposalId', pid, 'by', p_user, 'meetAt',
    p_meet_at));
  RETURN pid;
END $$;

-- FR-DLV-04: only the other party accepts or declines; returns the resulting status
CREATE FUNCTION delivery.respond_meetup(p_proposal uuid, p_user uuid, p_accept boolean)
RETURNS delivery.proposal_status LANGUAGE plpgsql AS $$
DECLARE h delivery.handovers; p delivery.meetup_proposals;
BEGIN
  SELECT h0.* INTO h FROM delivery.handovers h0
    JOIN delivery.meetup_proposals p0 ON p0.order_id = h0.order_id WHERE p0.id = p_proposal FOR UPDATE OF h0;
  IF NOT FOUND THEN RAISE EXCEPTION 'no␣such␣proposal' USING ERRCODE = 'check_violation'; END IF;
  SELECT * INTO p FROM delivery.meetup_proposals WHERE id = p_proposal FOR UPDATE;
  IF p_user NOT IN (h.buyer_id, h.seller_id) OR p_user = p.proposed_by THEN
    RAISE EXCEPTION 'only␣the␣other␣party␣can␣answer␣a␣proposal' USING ERRCODE = 'insufficient_privilege';
  END IF;
  IF p.status <> 'proposed' THEN RAISE EXCEPTION 'proposal␣is␣no␣longer␣open' USING ERRCODE = '
    check_violation'; END IF;
  IF h.status NOT IN ('arranging', 'agreed') THEN
    RAISE EXCEPTION 'a␣meetup␣cannot␣be␣arranged␣now␣(%)', h.status USING ERRCODE = 'check_violation';
  END IF;
  IF p.meet_at <= now() THEN
    UPDATE delivery.meetup_proposals SET status = 'lapsed', responded_at = now() WHERE id = p_proposal;
    RETURN 'lapsed';
  END IF;
  IF NOT p_accept THEN
    UPDATE delivery.meetup_proposals SET status = 'declined', responded_at = now() WHERE id = p_proposal;
    RETURN 'declined';
  END IF;
  UPDATE delivery.meetup_proposals SET status = 'superseded'
   WHERE order_id = p.order_id AND status = 'accepted';
  UPDATE delivery.meetup_proposals SET status = 'accepted', responded_at = now() WHERE id = p_proposal;
  UPDATE delivery.handovers SET status = 'agreed', agreed_proposal_id = p_proposal WHERE order_id = p.
    order_id;
  INSERT INTO delivery.outbox (event_type, aggregate_id, payload)
```

```
  VALUES ('delivery.meetup.agreed', p.order_id, jsonb_build_object('proposalId', p_proposal, 'pointId', p.
    point_id, 'meetAt', p.meet_at));
  RETURN 'accepted';
END $$;

-- FR-DLV-06: the seller enters the code; the service has compared it with the derived code in
-- constant time and passes the result. Returns 'accepted', 'wrong' or 'locked'.
CREATE FUNCTION delivery.record_code_entry(p_order uuid, p_seller uuid, p_correct boolean, p_max_wrong
    integer)
RETURNS text LANGUAGE plpgsql AS $$
DECLARE h delivery.handovers;
BEGIN
  IF p_max_wrong NOT BETWEEN 3 AND 10 THEN
    RAISE EXCEPTION 'CP-45␣must␣be␣between␣3␣and␣10' USING ERRCODE = 'check_violation';
  END IF;
  SELECT * INTO h FROM delivery.handovers WHERE order_id = p_order FOR UPDATE;
  IF NOT FOUND THEN RAISE EXCEPTION 'no␣handover␣for␣this␣order' USING ERRCODE = 'check_violation'; END IF;
  IF p_seller <> h.seller_id THEN
    RAISE EXCEPTION 'only␣the␣seller␣enters␣the␣handover␣code' USING ERRCODE = 'insufficient_privilege';
  END IF;
  IF h.status = 'locked' THEN RAISE EXCEPTION 'code␣entry␣is␣locked' USING ERRCODE = 'check_violation'; END
    IF;
  IF h.status NOT IN ('arranging', 'agreed') THEN
    RAISE EXCEPTION 'handover␣is␣closed' USING ERRCODE = 'check_violation';
  END IF;
  IF now() >= h.due_at THEN
    RAISE EXCEPTION 'the␣handover␣deadline␣has␣passed' USING ERRCODE = 'check_violation';
  END IF;
  INSERT INTO delivery.code_entries (order_id, seller_id, code_version, correct)
  VALUES (p_order, p_seller, h.code_version, p_correct);
  IF p_correct THEN
    UPDATE delivery.handovers SET status = 'completed', completed_at = now() WHERE order_id = p_order;
    UPDATE delivery.meetup_proposals SET status = 'superseded', responded_at = now()
     WHERE order_id = p_order AND status = 'proposed';
    INSERT INTO delivery.outbox (event_type, aggregate_id, payload)
    VALUES ('delivery.handover.completed', p_order, jsonb_build_object('at', now()));
    RETURN 'accepted';
  END IF;
  IF h.wrong_entries + 1 >= p_max_wrong THEN
    UPDATE delivery.handovers SET wrong_entries = wrong_entries + 1, status = 'locked', locked_at = now()
     WHERE order_id = p_order;
    INSERT INTO delivery.issues (order_id, kind, detail)
    VALUES (p_order, 'code_locked', jsonb_build_object('wrongEntries', h.wrong_entries + 1))
    ON CONFLICT DO NOTHING;
    INSERT INTO delivery.outbox (event_type, aggregate_id, payload)
    VALUES ('delivery.handover.locked', p_order, jsonb_build_object('wrongEntries', h.wrong_entries + 1));
    RETURN 'locked';
  END IF;
  UPDATE delivery.handovers SET wrong_entries = wrong_entries + 1 WHERE order_id = p_order;
  RETURN 'wrong';
END $$;

-- operator unlock after speaking to both parties: a new code, counter reset (audited by the service)
CREATE FUNCTION delivery.unlock_code(p_order uuid) RETURNS smallint LANGUAGE plpgsql AS $$
DECLARE h delivery.handovers;
BEGIN
  SELECT * INTO h FROM delivery.handovers WHERE order_id = p_order FOR UPDATE;
  IF NOT FOUND OR h.status <> 'locked' THEN
    RAISE EXCEPTION 'handover␣is␣not␣locked' USING ERRCODE = 'check_violation';
  END IF;
  UPDATE delivery.handovers
     SET status = CASE WHEN agreed_proposal_id IS NULL THEN 'arranging'::delivery.handover_status ELSE '
     agreed' END,
         locked_at = NULL, wrong_entries = 0, code_version = code_version + 1
   WHERE order_id = p_order;
  RETURN h.code_version + 1;
END $$;

-- FR-DLV-10 to FR-DLV-12: apply a courier status. Duplicates and out-of-date reports have no
-- effect; a report that skips 'picked_up' implies it. The service acts on the flags returned
-- (EscrowCommands calls, return request after a cancelled order) in the same transaction.
CREATE FUNCTION delivery.apply_status(p_shipment uuid, p_status delivery.shipment_status, p_occurred
    timestamptz,
                                      p_source delivery.status_source, p_event_id varchar, p_return_charge
    bigint DEFAULT NULL)
```

```
RETURNS TABLE (outcome text, pickup_recorded boolean, pickup_on_time boolean, delivered boolean,
               returned boolean, after_cancel boolean)
LANGUAGE plpgsql AS $$
DECLARE
  s delivery.shipments; t timestamptz; n integer; cur delivery.shipment_status;
  implied boolean := false;
  rank_of CONSTANT jsonb := '{"preparing":0,"booked":1,"picked_up":2,"in_transit":3,"delivery_failed":3,
␣␣␣
```

```
  IF s.order_cancelled_at IS NOT NULL AND (implied OR p_status = 'picked_up') THEN
    INSERT INTO delivery.issues (order_id, kind, detail)
    VALUES (s.order_id, 'pickup_after_cancel', jsonb_build_object('shipment', s.id)) ON CONFLICT DO NOTHING;
  END IF;
  INSERT INTO delivery.outbox (event_type, aggregate_id, payload)
  VALUES (CASE WHEN s.direction = 'return' AND p_status = 'delivered' THEN 'delivery.return.delivered'
               ELSE 'delivery.shipment.status_changed' END,
          s.order_id, jsonb_build_object('shipment', s.id, 'direction', s.direction, 'status', p_status,
                                         'simulated', s.simulated, 'at', t));
  RETURN QUERY SELECT 'applied',
         (implied OR p_status = 'picked_up'),
         (s.direction = 'forward' AND s.picked_up_at <= s.pickup_due_at),
         p_status = 'delivered', p_status = 'returned_to_seller', s.order_cancelled_at IS NOT NULL;
END $$;

-- FR-DLV-11: a not_received dispute may be opened once CP-17 has passed since pickup without delivery
CREATE FUNCTION delivery.not_received_allowed(p_order uuid, p_cp17 interval) RETURNS boolean
LANGUAGE sql STABLE AS $$
  SELECT EXISTS (SELECT 1 FROM delivery.shipments
                 WHERE order_id = p_order AND direction = 'forward'
                   AND status IN ('picked_up', 'in_transit', 'delivery_failed')
                   AND picked_up_at + p_cp17 <= now())
$$;

-- ====================================================================
-- Reference data: fare table version 1 (CP-43), from a published rate card (R14):
-- RedX merchant rates, inside Dhaka / sub-area / outside Dhaka: BDT 50 / 80 / 100 up to 1 kg,
-- 65 / 95 / 120 up to 2 kg, then 15 / 15 / 20 per kg. Administrators publish new versions.
-- ====================================================================
INSERT INTO delivery.fare_tables (version, effective_from, source)
VALUES (1, '2026-10-01T00:00:00+06:00', 'RedX␣published␣merchant␣rates␣(SRS␣R14),␣retrieved␣October␣2026;␣to␣
    be␣reviewed␣by␣the␣administrator');
INSERT INTO delivery.fare_rows (version, zone, up_to_1kg, up_to_2kg, per_extra_kg) VALUES
  (1, 'inside_dhaka',   5000,  6500, 1500),
  (1, 'dhaka_suburbs',  8000,  9500, 1500),
  (1, 'outside_dhaka', 10000, 12000, 2000);

-- ====================================================================
-- Grants (SEC-INF-03)
-- ====================================================================
GRANT USAGE ON SCHEMA delivery TO tb_app;
GRANT SELECT, INSERT, UPDATE ON ALL TABLES IN SCHEMA delivery TO tb_app;
REVOKE UPDATE ON delivery.code_entries, delivery.shipment_history, delivery.fare_tables, delivery.fare_rows
    FROM tb_app;
GRANT DELETE ON delivery.provider_events TO tb_app;                          -- retention CP-32
```